

City University of Hong Kong

Department of Data Science

# Obstacle-Aware Box Towing and Navigation for Quadruped Robots in Complex Terrain Using Reinforcement Learning

Lee Chung Hei

(Student ID: 57844011)

**Supervisor(s):** Prof. HO Chin Pang

**Submission Date:** 4<sup>th</sup> April, 2026

Submitted in part fulfilment of the requirements for the degree of

**Bachelor of Science in Data and Systems Engineering**

of City University of Hong Kong, 2026

# Project Team & Statement of Contributions

## Team Composition

| Role             | Student Name    | Student ID |
|------------------|-----------------|------------|
| Student (Author) | [Siu Nok Man]   | [57543816] |
| Teammate 1       | [Tsang Hin Man] | [57844938] |
| Teammate 2       | [Lee Chung Hei] | [57844011] |

## Statement of Contributions

### Contribution of [Siu Nok Man]

Literature review, validation on environment, towed payload system implementation, URDF, reward and cost functions optimization, validation planning and deployment preparation, Reset policy adjustment

### Contribution of [Tsang Hin Man]

Literature review, terrain modification, navigation strategy adjustment, observation space modification, reward and cost functions optimization, locomotion stabilization, obstacle avoidance optimization, policy validation and evaluation, deployment preparation

### Contribution of [Lee Chung Hei]

Literature review, sampling points to Grid, reward function optimization for payload towing, complex terrain implication, deployment preparation

# Declaration of Originality

I, Lee Chung Hei declare that the research presented in this thesis is my own work, except where acknowledged. The individual contributions of team members are clearly identified in the Statement of Contributions.

Student Name: Lee Chung Hei

Student ID: 57844011

Signature:      Lee Chung Hei\_\_\_\_\_ Date:4/4/2026 \_\_\_\_\_

# Ethics, Data Privacy, and Responsible Use

## Statement

Only open-source and publicly available tools which can be legally used by researchers will be used while working on this project, such as Isaac Gym (IsaacGym), Legged Gym (LeggedGym), and the Unitree Go2 URDF model (URDF Model). All these tools can be used to conduct research while respecting the rights of others to use them as they are permissively licensed. Throughout this project there was no any, there was no attempt to gather data from any person or gain access to or share private data with anyone. All simulation environments and policies developed were intended for academic research purposes only. The authors will share openly and fairly with the community in an open and transparent manner and avoid violating privacy or ethics.

# Abstract

Quadrupedal robots have proven to be highly competent at moving through challenging environments, but their practical application continues to be limited by their inability to perform the transportation of payloads in a wide variety of environments containing obstacles and clutter. This project focuses on addressing the challenge of enabling quadrupedal robots to tow box-shaped loads through complex terrain by utilizing a learning-based approach. We are developing an integrated reinforcement learning (RL) framework that combines goal-driven navigation with an obstacle-aware approach to achieving payload traction. The framework will use a binary grid-based terrain model for labeling nearby obstacles using local height measurements in order to support efficient copying and extensibility of spatial reasoning without requiring heavy perception systems. The RL control framework consists of two complementary components; the navigation component interprets the terrain structure to navigate the robot to its target while avoiding collisions with obstacles, and the traction component accounts for the kinematic constraints of the towed box to preserve cargo safety and stability during transport. We are also developing structured terrain benchmarks to measure and test the generalization of our RL framework across various types of obstacles. Together, these elements form a foundation for creating intelligent and robust locomotion control policies for robots capable of performing actual transport tasks in challenging real environments.

Keywords: Quadruped Robotic Locomotion, Reinforcement Learning, Payload Towing System, Obstacle Avoidance Navigation

# Acknowledgements

We want to say thank you to Professor Ho Chin Pan, Dr. Shaohang Xu and Mr. Yeung Chung Yui for their support and assistance during our project.

Over the course of the project, Professor Ho has met with us regularly to review our progress and give us feedback on what we needed to change and improve. Each month when we met, he was able to offer us new insights that shaped the way we approached our project and helped us understand more about it.

During the initial phases of our project, Dr. Shaohang Xu played an integral role in helping us decide which topic to pursue and where to go from there. His advice was also very instrumental in providing us with guidance and direction as we ran into difficulties in developing our concepts or making important design decisions.

Finally, we would like to extend our appreciation to Mr. Yeung Chung Yui for providing hands-on support through the technical aspects of our project. Mr. Yui was able to assist us in every aspect, providing practical advice with regard to deploying our products. His knowledge and expertise assisted us in overcoming many of the challenges that we encountered during our development cycle.

Together, Professor Ho, Dr. Xu and Mr. Yui provided us with a tremendous amount of knowledge and experience that resulted in both an intellectually stimulating and personally rewarding project.

# Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Declaration of Originality .....</b>                          | <b>2</b>  |
| <b>Ethics, Data Privacy, and Responsible Use Statement .....</b> | <b>3</b>  |
| <b>Abstract .....</b>                                            | <b>4</b>  |
| <b>Acknowledgements .....</b>                                    | <b>5</b>  |
| <b>Contents .....</b>                                            | <b>6</b>  |
| <b>List of Figures .....</b>                                     | <b>8</b>  |
| <b>List of Tables .....</b>                                      | <b>9</b>  |
| <b>1. Introduction .....</b>                                     | <b>10</b> |
| 1.1 Background .....                                             | 10        |
| 1.2 Problem Statement and Objectives .....                       | 11        |
| <b>2. Literature Review .....</b>                                | <b>14</b> |
| 2.1 Summary of Key Findings .....                                | 14        |
| <b>3. Methodology .....</b>                                      | <b>17</b> |
| 3.1 Pipeline .....                                               | 17        |
| 3.2 Modeling Strategy .....                                      | 19        |
| 3.2.1 Reinforcement Learning .....                               | 19        |
| 3.2.2 Legged Gym Framework .....                                 | 20        |
| 3.2.3 Locomotion .....                                           | 21        |
| 3.2.4 Observation .....                                          | 35        |
| 3.2.5 Payload Towing System .....                                | 38        |
| 3.3 Evaluation Metrics and Validation Plan .....                 | 42        |
| 3.4 Validation Plan .....                                        | 45        |
| <b>4. Experiments and Results .....</b>                          | <b>49</b> |
| 4.1 Ablation / Sensitivity Analysis .....                        | 49        |
| 4.1.1 Sensitivity to Command Resampling Interval .....           | 49        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.1.2 Sensitivity to Target-reaching Threshold .....              | 50        |
| 4.1.3 Sensitivity to Collision Detection Threshold .....          | 50        |
| 4.1.4 Sensitivity to Box Mass.....                                | 51        |
| <b>5. Discussion .....</b>                                        | <b>52</b> |
| 5.1 Limitations and Practical Implications.....                   | 52        |
| 5.2 Practical Implications .....                                  | 53        |
| <b>6. Individual Technical Contributions: Lee Chung Hei .....</b> | <b>55</b> |
| 6.1 Overview of Responsibility.....                               | 55        |
| 6.2 Implementation Details.....                                   | 56        |
| 6.3 Critical Analysis .....                                       | 59        |
| <b>7. Conclusion and Future Work .....</b>                        | <b>60</b> |
| 7.1 Summary.....                                                  | 60        |
| 7.2 Future Work .....                                             | 61        |
| <b>Bibliography.....</b>                                          | <b>64</b> |
| <b>Appendix .....</b>                                             | <b>66</b> |
| A.1 GitHub repository of the project .....                        | 66        |



# List of Figures

|                   |    |
|-------------------|----|
| Figure 3.1 .....  | 26 |
| Figure 3.2 .....  | 26 |
| Figure 3.3 .....  | 28 |
| Figure 3.4 .....  | 32 |
| Figure 3.5 .....  | 37 |
| Figure 3.6 .....  | 39 |
| Figure 3.7 .....  | 42 |
| Figure 3.8 .....  | 43 |
| Figure 3.9 .....  | 43 |
| Figure 3.10 ..... | 45 |
| Figure 3.11 ..... | 47 |
| Figure 3.12 ..... | 47 |

# List of Tables

|                 |    |
|-----------------|----|
| Table 4.1 ..... | 49 |
| Table 4.2 ..... | 50 |
| Table 4.3 ..... | 50 |
| Table 4.4 ..... | 51 |

# 1. Introduction

## 1.1 Background

In recent years, due to their ability to navigate through challenging terrains where wheeled/tracked-based perspectives and robotics cannot, quadruped robots have become increasingly popular. In order to get through these areas successfully, they use their four limbs to make it possible for them to traverse uneven ground, climb things over and maintain stability on irregular ground. With this technology available, there are many different uses for quadruped robotics; including but not limited to disaster response, inspection, exploration, and logistics. With advancements being made in developing more sophisticated and functional robotic mobility systems, interest continues to grow in expanding quadruped robotic capabilities from merely the basics of locomotion to being able to conduct a wide variety of practical operations within the real world.

The ability to move payloads is a critical physical function that is required for most types of robotics. They must not only be able to move themselves but must also be able to move tools, equipment, or materials from one place to another. For example, robots used in disaster relief operations may deliver supplies through an area with debris, while robots working in construction or logistics would be required to help transfer equipment across different types of terrain. Therefore, enabling quadruped robots to perform payload transportation will greatly enhance the functional potential of quadruped robots.

In contrast to standard locomotion tasks, payload transport poses new problems. A robot's interaction with an external object creates a unique set of dynamically coupled motions that can affect the robot's stability while walking and navigating toward a designated goal. The payload's relative position to the surrounding environment must also remain discrete from obstacles during navigation, and ensure that the transported object does not collide with the surrounding environment.

The towing of payloads is a straightforward and user-friendly method of moving payloads for quadrupedal robots. This is accomplished by connecting to the payload using a tether or some other type of link or coupling so that the robot can then tow the payload without requiring any complex mechanical manipulation to move the payloads. However, when a quadrupedal robot tows a payload, there are new interactions between the robot and the payload that may impact the gait stability of the robot and its ability to navigate effectively. Therefore, this provides the motivation to research and develop methods that enable quadrupedal robots to safely and efficiently tow a payload in complex environments.

## 1.2 Problem Statement and Objectives

This study is inspired by the disconnection between the theory behind current approaches to multi-robot transporting and the practical limitations that will be experienced when these multi-robot systems are used in the real world. Although some ideas have shown that it is possible to accomplish a task under controlled conditions, it is becoming increasingly less likely that they will be able to accomplish this task in a scenario that is truly representative of the conditions under which they would normally be used, such as restricted spatial access to the area where a payload is located, a failure of the communications infrastructure, or limited availability of robotic assets. Environments of disaster typically have tight corridors that prohibit multiple robots from operating simultaneously. A military operation may require a stealth movement where any form of inter-robot communication will compromise the tactical location of any involved robotic assets, and an operation in a resource-limited venue may have a lack of robotic resources entirely. Therefore, it may be a vital operational requirement to have one quadrupedal robot be able to independently transport a payload through terrain that contains a high density of obstacles, which is not being satisfactorily addressed by currently existing approaches.

Making payload towing with obstacle-aware navigation into existing conventional locomotion frameworks presents a technical challenge, as the original designs of these frameworks did not consider it to be part of their primary purpose. While most current control methodologies for quadruped robots will use either a stable locomotion generation method or a high-level path planning method, there are very few methods that are able to take both of these aspects. Furthermore, there are very few approaches that have been able to consider the dynamic disturbances caused by the presence of the towed payload. When a quadruped robot tows a payload, the coupled system exhibit modified dynamics that can destabilize gaits optimized for unloaded operation. In addition to maintaining balance while proceeding toward the specified goal, avoiding obstacles in the path created by the environment where the robot travels, and making sure that the trailing payload does not hit any structures that surround the robot, it is still a major challenge to coordinate these conflicting objectives into a single unified control system.

The technical challenges of integrated payload towing and obstacle aware navigation increase the challenges above for traditional locomotion systems that were not designed with these integrated functions. For instance, while many existing control approaches for exploiting the stability of quadrupedal gaits and the associated dynamic instability of a payload being towed are illustrated, there is little indication that either of these approaches normally considers the ability of the quadrupedal robot to maintain its balance when towing the payload, while also maintaining forward progress towards its intended destination, while avoiding any obstacles in its path and avoiding the probability of collision

between the trailing payload and its environment. Thus far, there has been no successful approach to resolving how to unify these functional and competing unifying objectives within a single control and coordinated framework.

In addition, conventional perception pipelines impose a high computational burden on an embedded platform, limiting deployment options. These two powerful environmental representations, dense elevation maps and point cloud representations, are typically too computationally intensive to run on the embedded systems commonly used on legged platforms. As a result, the computational load creates an increased latency, significantly reduces the usable runtime of the battery, and drastically reduces the ability to operate in real time. Therefore, if new perception methods were developed that could retain sufficient navigation information while eliminating at least some of the computational costs, the practicality of deploying them would increase.

To address these identified gaps, the proposed solution developed a novel reinforcement learning framework that combines the various elements of stable locomotion, obstacle-aware navigation, and payload towing into a single unified control policy. As a means of encoding environmental knowledge, a binary grid representation with respect to obstacle presence was employed in a local grid that encompassed the robot. The use of a binary representation reduced the number of perceived environmental variables from thousands of floating-point numbers to hundreds of binary numbers, thereby decreasing computational demand while preserving sufficient environmental spatial knowledge for the purpose of avoiding collisions. The learning process was developed based upon the principles of constrained Markov decision theory (Altman, 1999) and was maximized through the use of Constrained Proximal Policy Optimization (Achiam, J., Held, D., Tamar, A., & Abbeel, P. 2017), resulting in the simultaneous maximization of task reward and compliance with the constraints associated with safety, falls, joint constraints, and box collisions.

The specific objectives of this research are stated below. First, to develop an integrated reinforcement learning based framework that can teach a single quadruped robot simultaneously towing a box while obstacle-aware navigating a course is the first objective. Second, to implement and verify a computationally efficient binary grid based obstacle representation for environmental perception, with minimal computation overhead. Third, to develop and systematically tune the reward functions, which balance competing objectives of locomotion stability, navigation accuracy, and payload safety throughout the training phases. Fourth, to train the resulting policy and evaluate the performance across multiple terrain types of simulation, i.e. sparse obstacle fields, cluttered environments, and uneven terrain with varying elevation. Fifth, to deploy the trained policy onto physical hardware with no additional fine-tuning or adaptation procedures. Sixth, to quantify system performance using objective metrics including navigation success rate, box collision

frequency, distance travelled, robot fall rate, and goal reach accuracy. Once these objectives are met, it will be established if single-agent payload towing is a viable alternative to multi-robot systems in operationally constrained environments.

## 2. Literature Review

### 2.1 Summary of Key Findings

Early work in quadrupedal locomotion demonstrated the potential of end-to-end deep RL to achieve agile, robust movement on challenging terrain. For example, Lee et al. (2020) trained a neural controller using only proprioceptive inputs, producing a “radically robust” ANYmal locomotion policy that transferred from simulation to real-world obstacles, such as mud, snow, and rubble, without any manual tuning. In the same year, Yang et al. (2020) introduced a multi-expert RL architecture that blends several pre-trained locomotion policies via a gating network, allowing a quadruped to adaptively switch among trotting, steering, and even fall-recovery behaviors when facing new scenarios.

Model-free RL initially created the foundation for developing autonomous controllers that could function as efficiently as any animal through determining the best rewards through simulation-based experiments. The ability for models to be trained on large amounts of data was made possible by using GPUs – for instance, Rudin et al. (2022) developed good ANYmal walking policies in Nvidia's Isaac Gym environment in a matter of minutes (~4 mins on flat surfaces; ~20 mins on uneven surfaces) by taking advantage of the ability to simultaneously simulate thousands of robots. The acceleration of this process is orders of magnitude faster than previous methods of research in the area of modelling legged locomotion controllers, thus providing a solid foundation for developing controllers that exhibit more complex behaviours.

### Integrating Vision and Hierarchical Planning

Researchers began integrating richer perception and high-level planning into locomotion policies. Miki et al. (2022) addressed the challenge of using vision for legged robots: they designed an attention-based recurrent network that fuses exteroceptive (depth) data with the robot's proprioceptive state, modulating the gait to yield a controller that is both fast and robust in the wild. Their “robust and general” solution was validated on an hour-long real-world hike in the Alps, combining proprioception and vision allowed the quadruped to maintain stability across snow, vegetation, steep stairs, and low-light conditions.

Similarly, Hoeller et al. (2024) presented a hierarchical RL approach for agile navigation. They trained individual locomotion “skills” (walking, jumping, climbing, crouching) in simulation on different obstacle types, then learned a high-level policy that uses a learned perception module to plan sequences of these skills through cluttered terrain.

This ANYmal “parkour” system reconstructs obstacles from sparse sensor data and autonomously decides which locomotion skill to deploy. In hardware tests, it successfully traversed consecutive challenging obstacles at speeds up to 2 m/s without human intervention.

Jenkins et al. (2024) present a hybrid method known as "Deep Tracking Control", which leverages a model-based planner to produce reference foot-position trajectories while also letting a neural policy rapidly learn to replicate the same trajectories. Together, these two strategies provide a much improved level of performance for the controller when traversing a sparse, uneven surface where the most likely optimal foothold (or foot placement) position is known - although the most likely optimal plan also provided more effective performance than the model based planner under slippery/deformable conditions. These findings reflect a marked increase in research interest in combining RL-based locomotion control with perception/decision-making capabilities and the introduction of hierarchical decision-making to enable intelligent obstacle avoidance in real time as opposed to simply following a defined series of velocity commands.

## **Obstacle Avoidance and Environmental Mapping**

Obstacle avoidance and environmental mapping are naturally integrated into RL frameworks. Hoeller et al.'s hierarchical policy implicitly integrates locomotion and navigation since it uses learned obstacle reconstructions to guide the selection of skills. The DTC method developed by Jenelten et al., on the other hand, uses the geometry of the terrain to explicitly plan the placement of footfalls so that the controller naturally avoids large gaps and steps. Other efforts simplify the perceptual input by feeding a compact binary grid map of obstacle locations directly into the policy, preserving essential spatial structure while greatly reducing observation dimensionality. The main takeaway is that any type of structured representation of the environment, whether it is learned through RL or designed with human input, will assist RL agents in their reasoning about the geometry of obstacles. Thus, recent works in navigation-oriented RL demonstrate that the combination of predictive mapping/planning and adaptive control allows quadrupeds to clearly identify and circumvent obstacles in real time.

## **Payload Transport and Towing: A Largely Unaddressed Challenge**



Payload transport and towing are still under-researched in regard to basic single robot RL algorithms, since all previous works have either utilized on only aerial-based vehicles or multiple robot teams. In the case of Chen et al. (2025), they investigated decentralized multi-agent RL with a team of quadrupeds cooperatively towing a payload via a cable. Ji et al. (2021) investigated using cooperative RL between two quadrupedal robots manipulatively traversing a payload across rough terrain. Both studies assume that their robot team is working together, and as such do not discuss how to deal with pulling an object through a field of obstacles; thus, the challenge of having only one quadrupedal robot pulling a box in an area with a significant amount of obstacles is missing from the existing literature and needs further investigation.

This underexplored task couples locomotion stability, payload dynamics, and navigation. Our work draws on existing insights but specifically tackles this gap with an end-to-end learning solution for obstacle-aware single-robot towing. Solving such a problem requires the quadrupedal agent to simultaneously reach target goals, avoid collisions for both itself and the towed box, and maintain a stable gait under the added load.

## **Safety-Constrained Reinforcement Learning for Locomotion**

An increasing number of researchers have begun to pay attention to the issue of safe and trustworthy learning. Constrained Reinforcement Learning (RL) is a framework that has been developed for this purpose by incorporating safety constraints along with the standard RL objective of maximizing reward. The structure of the Constrained Markov Decision Process (CMDP) as presented by Altman represents a foundation for all modern safety-focused RL methods. Building on this foundation, Achiam et al.'s 2017 work pioneered Constrained Policy Optimization (CPO), an algorithm that enforces safety by maintaining near-constraint satisfaction at each policy update. Gangapurwala et al. (2020) applied a “Guided CPO” variant in which an ANYmal robot learns to track commanded velocities while respecting foot-contact and kinematic limits.. The CPO method allowed training a controller to ensure stability over an extended period of time without the need for extensive tuning of the reward function. Notably, the performance results of the Guided CPO trained robot were closely aligned with the performance results obtained using unconstrained RL methods while consistently resulting in safe and feasible behavior. Through these advancements in the area of robot locomotion, it has been shown that adding explicit constraint terms to the RL objective can lead to policies that promote safe, stable, and collision-free robot locomotion.

In sum, the field has progressed from early locomotion skills on flat, simple terrain to today's robust, multimodal controllers with richer perception, hierarchical planning, and integrated safety mechanisms. These advances now enable quadruped robots to purposefully navigate and even tow payloads through complex, obstacle-filled environments.

## 3. Methodology

### 3.1 Pipeline

The Legged Gym simulation system is the foundation for the pipeline of the project. This simulation system is powered by the Isaac Gym physics engine, which enables the creation of high-performance GPU-accelerated simulations (Makoviychuk et al., 2021). The pipeline of the project consists of several components, including environment setup, representation of robots and their payloads, modeling of terrains, representation of perception data, and data collection methods.

As the core of all training and evaluation activities, the Legged Gym framework will facilitate thousands of environments with one GPU, providing an improvement to the speed at which data can be collected. This methodology is consistent with established practices in legged locomotion research and will allow for realistic actuator dynamics to be included within the framework of the project. The Isaac Gym API can provide mechanisms for collision detection, rigid body dynamics, and sensor simulation; a demonstration of that is how the code uses gymapi functions extensively to manage tensors and refresh state.

This project will utilize the Unitree Go2 quadruped robot platform, which can be imported as a URDF (Unified Robotics Description Format) asset file from a directory path based on the project's primary directory. This robot has a total of 12 degrees of freedom (DoF), comprising one DoF for each hip, thigh, and calf joint on the four legs. The way in which joints are ordered on this robot adheres to Isaac Gym's set of conventions about how to label the robot's joints. Each of the feet has a force sensor attached that accurately measures the forces that the robot is applying to the ground when it is in contact with the ground, and an optional depth camera can be attached to the robot for visual perceptual tasks. The physical characteristic of the robot (mass, mass distribution, and collision geometry) are defined in the asset file and can be modified by domain randomization parameters in the configuration file.

The payload that is pulled behind the robot is designed as a rigid cube box that has different sizes and weights, which can be adjusted based upon parameters contained within an appropriate configuration file. A box object is created procedurally using the `gym.load_asset` and the dimensions for the box are dictated from the specified parameters with the URDF file. The mass of the box is attached at the time of creating the actor by setting mass properties of the rigid body. Rather than using a real physical joint between the robot and the box, a rope chain consists of ten intermediate links connected by revolute joints is used to allow passive swinging. The final joint attaches the box to the end of this chain. As the box is associated with the same actor, no separate kinematic constraint or manual repositioning is required. The physics engine automatically accounts for all aspects of the entire articulated system including collision handling, joint dampening as well as real-world slack effects. Therefore, the location and orientation of the payload will also be automatically updated as they transition throughout simulated time and provide natural tension and following response in relation to the rope joints.

The terrain is generated procedurally using a dedicated Terrain class imported from the utils module. There are 3 different types, which are plane, heightfield, and triangle mesh. More complex environments will have more varied terrain topography with obstacles, based on heightfields or trimesh generation. Configuration details, including friction and restitution coefficients for the terrain dimension and property definitions will be provided when the simulation is created from a given configuration. The height samples will be saved in a tensor for future use during perception and rewards calculations. The terrain curriculum allows the difficulty to be progressively adjusted based on the performance of the robot, allowing staged learning from simple to complex environments.

One of the key contribution of this project is building a binary grid representation from sampled height data using the terrain height measurement locations around the robot. The `_init_height_points` method builds a series of sampling points around the robot base in a grid, and then the `_get_heights` function transforms those points into world space and interpolates the height values from the heightfield. The absolute height values are thresholded to identify cells as obstacles for the binary grid generation. For example, a minimum threshold 0.3 meters is used to classify the terrain cell as an obstacle. The resulting binary data not explicitly in the form of a tensor from the code will represent the obstacles that would fill the cells of the binary grid. This method reduces computation cost by not using dense elevation maps while keeping enough spatial information for navigation decisions.

The policy training runs with 4096 parallel environments as indicated by the `num_envs` parameter found throughout the code. Each environment has its copy of the robot and box to allow for many different experiences to be captured simultaneously. The step function handles processing actions from the policy, calculating torques using PD

controllers, advancing the physics simulation, and returning observation, reward and reset flag. The `compute_observations` method builds the observations by concatenating proprioceptive data, command data, and terrain height data. The typical reinforcement learning flow of gathering multiple transitions to be used for policy updates occurs during the experience collection process. History buffers are built for maintaining sequences of observations and actions for algorithms which require temporal context.

Several preprocessing steps are applied to the observations prior to passing them into the policy network, using scaling factors from the configuration to normalize the different sensor modalities and ensure similar ranges. All linear velocities are multiplied by `lin_vel` scale, all angular velocities are multiplied by `ang_vel` scale, and all joint positions are multiplied by `dof_pos` scale. Domain randomization can be utilized to introduce sensor noise to the observations when the `add_noise` flag is turned on (Tobin et al., 2017); The vector of noise can have a different scaling per observation modality to ensure that realistic imperfections in the sensors support the simulation. Finally, each observation will be clipped to a specified range to prevent outliers from destabilizing training. During training, privileged observations will provide the additional useful information such as mass parameters, coefficients of friction, and motor strength randomizations for use by the critic network to develop a more accurate value function.

## 3.2 Modeling Strategy

### 3.2.1 Reinforcement Learning

Reinforcement Learning (RL) is a type of Machine Learning that teaches an Agent how to make better decisions based upon information available through its constant interaction within an environment, in order to ultimately reach a particular set of Long-Term Objectives. At each step, the agent receives an observation (or state), selects an action, and receives feedback in the form of a reward signal. RL, therefore, does not have to utilize or incorporate any form of pre-existing Labeled Data to teach the Agent; this allows RL to focus on more complex tasks such as controlling Legged Motion and Path Navigation.

Formally, RL problems are modeled as **Markov Decision Processes (MDPs)**, represented by the tuple  $(S, A, P, R, \gamma)$ , where:

- $S$ : the set of all possible states the agent can observe,

- $A$ : the set of all possible actions the agent can take,
- $P$ : the transition function  $P(s' | s, a)$ , defining the probability of moving to a new state  $s'$  from state  $s$  after taking action  $a$ ,
- $R$ : the reward function  $R(s, a)$ , which gives feedback on the quality of the action,
- $\gamma$ : the discount factor, determining the importance of future rewards.

Reinforcement learning will be applied to train a quadruped mobile robot to transport an object through varied terrain containing obstacles. The robot acts as the agent and receives information about its current state, including its joint position(s), speed, terrain characteristics, and location of the target. Its actions are continuous torque commands applied to the joints. The agent gains experience in a simulation by repeatedly executing tasks and learning effective locomotion techniques and cargo transporting methods for maximum task completion with the least amount of injury to the agent. To mitigate potential risk of unintentional injury during training, a method known as CPPO (Constrained PPO) will be described later as one possible solution to constrain the risk of unsafe behaviour during RL training.

### 3.2.2 Legged Gym Framework

The legged gym serves as the simulation foundation for the entire project. It provides a very high throughput environment enabling researchers to apply reinforcement learning techniques to train quadrupedal robots. Built on NVIDIA Isaac Gym, it supports massively parallel simulation, allowing users to run thousands of simultaneous environments for training on one single GPU. This allows for more data input and real-time monitoring of task successes and safety violations as well as provides the ability to support constraint aware learning.

By extending the legged gym framework to create a new environment known as Go2Env, we have created a custom environment for training a four-legged robot while towing a dynamic payload across varied terrain. Every simulation cycle involves observing proprioceptive and exteroceptive sensory input data, applying the appropriate joint torque as determined by the interaction with the learned behaviour policy, and receiving both the reward and constraint signal from the environment. The environment supports both stochastic terrain collection and a fixed map

mode and can therefore be used to develop and validate the performance of the legged robots trained under randomised conditions and against consistently reproducing test cases.

Using the `terrain.py` module, the terrain system divides the simulation space into a 10×20 grid of sub-terrains, each measuring 8×8 m. The columns vary in terrain type, e.g., smooth surfaces, rough surfaces, stairs and discrete obstacles, while the horizontal rows represent an increase in the level of challenge in the terrain segments, which creates a progression of difficulty much like a curriculum framework would allow. Between each of the sub-terrain segments there are flat buffer zones that assist with gait recovery as the robot moves through the environments. The Fixed Mode uses a fixed pattern of obstacles that have been arranged to allow the user to configure the level of challenge and width of gaps, so the user's ability to assess the effectiveness of their robotic policies will be the same across multiple attempts to evaluate them. These fixed maps are particularly useful for constraint tuning and benchmarking failure modes.

At Legged Gym, we have designed a platform that provides developers with full access to simulator low-level physics state for developing detailed reward functions based on things like foot placement accuracy, gait timing, payload alignment, and torque efficiency. We've also built the environment so that it can log all constraint violations and safety metrics online, facilitating reliable use of constrained policy optimization.

Overall, legged Gym is an excellent example of a scalable, customisable, and observable environment that fully supports our end-to-end learning pipeline. Through the use of Legged Gym, we are able to use simulation to develop robots with very complex, stable locomotion and tasking behaviours, while also designing the simulation with sufficient fidelity and control transparency for transfer to the real world later on.

### **3.2.3 Locomotion**

Locomotion refers to the ability of a robot to move itself from one location to another by coordinating its body and limbs. In the context of legged robotics, it involves generating stable and efficient motion patterns—called gaits—by controlling the movement of each leg. Effective locomotion requires dynamic balance, rhythm, and adaptability to external forces and terrain conditions.

In this project, locomotion is a core capability that allows the quadruped robot to traverse complex environments while towing a payload. Rather than using hand-crafted control rules, the robot learns locomotion behavior through reinforcement learning.

## Constrained Proximal Policy Optimization (CPPO)

Constrained RL extends standard RL by adding safety-critical limits on behavior. In most cases, determining a specific reward & separate constraint costs (for example, safety and stability) makes constrained rewards easier than defining everything in the reward function. In a Constrained MDP (CMDP) setting we maximize the expected discounted return  $J(\pi) = E_{\pi}[\sum_t \gamma^t r_t]$  subject to expected cost constraints  $E_{\pi}[\sum_t \gamma^t c_t^i] \leq d^i$ . CPPO is a variant of PPO that enforces such constraints. An example of this application would be through quadruped locomotion, where the reward typically rewards the commanded base velocity while rewarding efficiency of motion. In contrast, cost functions consist of unsafe conditions associated with quadruped locomotion, including extreme joint velocities, the ZMP existing outside of the support polygon and animals experiencing a complete loss of contact with the surface they are walking on. By including constraints explicitly, CPPO avoids unsafe or unnatural gaits that ordinary PPO might learn.

## Reward and Cost Function Design

**Reward terms:** The locomotion reward system is composed of a variety of weighted terms that promote positive behaviors. In the ANYmal quadruped example, these include tracking forward linear velocity (via a logistic-shaped kernel on  $\|v - v_{des}\|$ ), penalizing excessive joint torques, yaw/angular velocity error, and undesirable foot motion. More broadly, other reward terms can be used to discourage foot slip, vertical foot accelerations that are larger than normal (thus leading to smoother and more efficient locomotion), and body orientations deviating from an upright stance. These serve as guiding principles in the development of the agent's learning. However, they are not strictly enforced in the learning process, e.g., the foot-clearance or smoothness rewards serve to direct the gait without being required to be satisfied strictly.

**Cost (constraint) functions:** The costs that must be satisfied by the policy represent the actual safety/stability constraints. Examples of hard constraints are limits on joint velocities, joint accelerations and criteria relating to stability. In GCPO, hard constraints were

implemented as cost returns  $E \sum \gamma^t c_t$  that must stay below a threshold. Concrete examples include: keeping each foot within a prescribed “eligible” support region, and ensuring the robot’s zero-moment point (ZMP) stays inside the support polygon, so it will not tip over. In the Definitions, the penalty costs were defined for simultaneous velocity/acceleration of the robot feet and simultaneous foot placements by the robot that exceeded their allowable operational ranges.

## CPPO Algorithm and Constraint Enforcement

CPPO modifies the PPO surrogate objective to enforce constraints. In one practical implementation (GCPO), training an approximate CPPO (aCPPO) phase where constraints are handled via penalties.

- aCPPO (approximate phase): We use the standard PPO clipped objective with Generalized Advantage Estimation (GAE) for reward, augmented by a cost penalty. Concretely, the surrogate loss is

$$L_{aCPPO}(\theta) = E_{(s,a)} \left[ \min(r(\theta)\widehat{A}_r \text{clip}(r(\theta), 1 - \varepsilon, 1 + \varepsilon)\widehat{A}_r) \right. \\ \left. - \alpha E(s, a) [\min(r_c(\theta)\widehat{A}_c, \text{clip}(r_c(\theta), 1 - \varepsilon, 1 + \varepsilon)\widehat{A}_c)] \right. \\ \left. + \beta H[\pi\theta] \right]$$

where  $r(\theta)$  are reward-advantage ratios and  $r_c(\theta)$  are cost-advantage ratios. In other words, the cost advantage  $\widehat{A}_c$  is subtracted (with weight  $\alpha$ ) similarly to PPO’s clipping. Intuitively, this phase treats the cost as a differentiable penalty: policies are nudged toward low cost but without a hard guarantee. This surrogate training is more sample-efficient than solving the constrained problem exactly.

## Lagrangian Multipliers and PPO Updates

A common alternative is a primal–dual (Lagrangian) method: one forms the Lagrangian  $\mathcal{L}(\theta, \lambda) = E[R] - \lambda(E[C] - d)$  and alternates gradient updates on the policy parameters  $\theta$  and the dual variable  $\lambda$ . Typically,  $\lambda$  is increased (via gradient ascent or even a PID controller) when the expected cost exceeds its limit, so as to penalize infeasibility. However, this introduces extra hyperparameters and often unstable convergence (the  $\lambda$



update behaves like an integrator). The approach proposed by Xuan et al. in the CPPO framework eliminates the need for explicit Lagrange multiplier parameters, and instead, casts a CMDP problem as a probabilistic inference problem. In effect, the E-Step is essentially an “optimal” policy within the safe set, and the M-Step is a first-order PPO-style update of the policy, to the optimal policy. The first-order nature of the CPPO update eliminates the need for second-order matrix inverses and tuning of the dual variable, making it less difficult to implement and more scalable than the primal-dual method.

Standard PPO (no constraints) can be used as a method to train a neural network, but it will only be optimizing the highest score possible without regard for safety. In contrast, CPPO incorporates safety into its training process by incorporating its costs (the amount of time spent outside the required speed/acceleration limit) into the reward process to encourage the agent to stay within these bounds while training. During quadruped experiments, it was found that normal PPO would converge on unsafe solutions (e.g. very high joint speeds) even if the rewards were tuned to be at optimal levels, however, using CPPO reduces violations of constraints from <1% with an unconstrained policy to only ~0.02–0.05% for the final policy. Thus, it can be concluded that CPPO effectively ensures that agents stay within the required speed or acceleration limits when training.

## **Stable and Safe Gait under Uncertainty**

CPPO utilised constraints during training along with domain randomisation for creating robust gaits for the robot. As part of CPPO's training, authors were also able to apply various forms of noise to the observations and actions in addition to varying the gravity and actuator gains, and scaling the link masses to simulate the uncertainty associated with physical hardware operation. One of the most significant aspects of the CPPO framework is that the interpretation of constraints through CPPO allowed the robot to operate within safe limits regards to these kinds of related perturbations. Therefore the learning policy from CPPO would guarantee that the robot always had two feet on the ground (no pronking/falling) and would always recover to a safe configuration regardless of the extent of a violation (the addition of a guided recovery reward added a reward for whenever the cost-constrained policy was re-satisfied).

The CPPO trained controller performed significantly better in both real and simulated tests than the unregulated PPO controller. The CPPO controller tracked commanded speeds on the real-life ANYmal robot more closely than any other controller (with the addition of the unforeseen payload), and it only required approximately 20 Nm of total joint torque compared to the traditional model-based trot controller. Additionally, throughout the 5-

minute test period, the CPPO controller consistently adhered to its maximum joint torque limits, only violating them 0.02-0.05% of the time as opposed to violating the limit 1.2% without CPPO.. Similarly, the CPPO controller produced stable gaits in the presence of uncertainty, allowing for random control delays of up to 20 ms, external forces themselves capable of producing 100 N of force (recovering upright after each push), the presence of one soft actuator on the robot, and even in a low-gravity environment ( $1.62 \text{ m/s}^2$ ) without having to retune the system.

In conclusion, compared with standard PPO methods, CPPO incorporates the cost constraints into the learning process before applying them to the current state of the agent. This is achieved without needing to utilize a complex method of implementing the second order method or having to enforce any manual reward design. Using the CPPO algorithm has allowed the agent to develop a high-performance gait that is both learned and demonstrated as safe by meeting the constraints placed on the kinematics of the joints as well as the support points, even under disturbances.

## **Terrain creation**

In this project, the terrain is part of the environment state within the reinforcement learning pipeline. The robot performs actions on different terrains and learn an optimal policy based on the rewards it receives.

The terrain in the training environment is divided into 20 columns and 10 rows of sub-terrains, each sub-terrain is 8x8 meters. The rows represent the level of difficulty, while the front rows have lower difficulty. The columns represent different sub-terrain types. The terrain types in the project includes:

- Smooth Slope: Continuous inclined planes with flat platforms
- Rough Slope: Sloped surfaces with random height variations
- Stairs Down: Descending steps
- Stairs Up: Pyramid-like stairs
- Discrete Obstacles: Flat ground with randomly placed rectangular obstacles

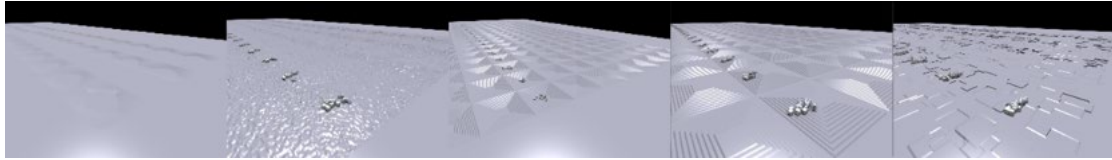


Figure 3.1: Different types of terrain configurations created and used to train the quadruped robot

The terrain type selection in each column is randomized based on the configuration. In this project, the terrain proportions are set to  $[0.1, 0.1, 0.35, 0.25, 0.2]$ , which means the terrain consists of 10% smooth slope and rough slope, 35% stairs down, 25% stairs up, and 20% discrete obstacles.

To facilitate stable training, a flat area is added between each sub-terrain. The flat area can act as a safe starting zone before entering the terrain of the next level.

In addition, multiple rectangular obstacles are placed in each sub-terrain randomly. If the positions are too close to the center and edge, they would be rejected and resampled again, the maximum resample attempt is 20 times. The number of obstacles is limit to 3 to 6 in each sub-terrain, which varies by the difficulty level of the terrain.

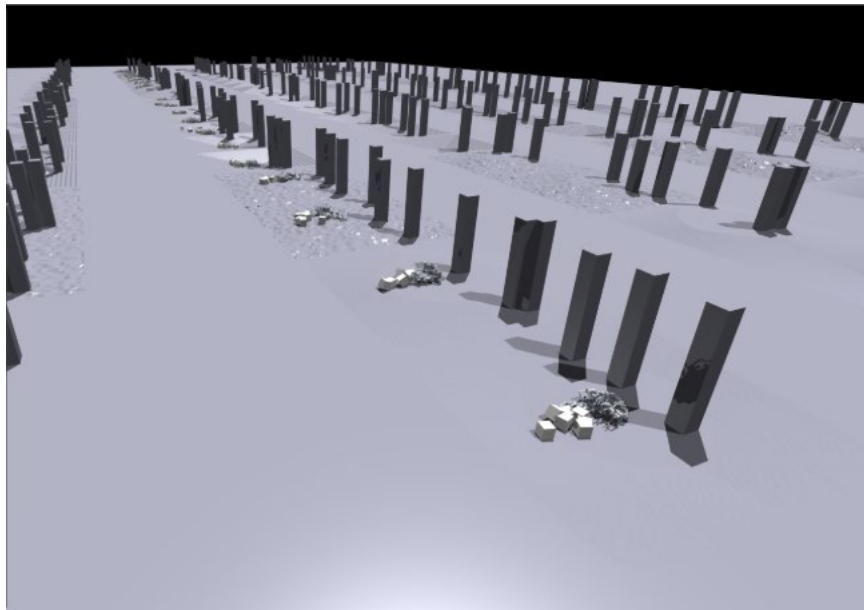


Figure 3.2: An image of the training environment, showing how the robot interacts with obstacles and various terrains during the simulation.

## Static Terrain Environment

In addition to the randomized terrains used during training, this project incorporates a static terrain mode to support structured evaluation and consistent benchmarking. In Static Terrain Mode, we will replace our randomised layouts with a static, set of pre-created Obstacle Maps that have been specifically created to test different types of locomotion and navigation challenges.

The maps will be fixed however, the level of difficulty will be different; therefore, the user will have a clear and reliable way of Comparing Performance across all evaluations. Each of the Static Maps represents an 8 m × 8 m Area. Each map will fall under the five basic Obstacle Patterns: Zigzag Walls, Scattered Pillars, Winding Corridors, Chicane, and Gap Walls. The number of obstacles laid out in each Pattern is based on the Column Number, while the level of Difficulty in each Pattern increases as the user moves across the Rows.

The following Key Parameters define the Maps. The Height of the Obstacles is set at 0.5 m; this height defines the obstacles as Non-Traversable and therefore will cause the Robot to need to look for Navigation Paths around them. The Minimum Passage Width between the Obstacles is set at 1.2 m; this gap is wider than the robot's width, therefore it allows for Navigation through constrained Passageways, but allows for successful Navigation through the obstacle Course. The Difficulty Scale has the capability of being adjusted between a range of 0.5 and 1.5, this Capability Controls the Density of Obstacles in the Map, provides additional Complexity in How the obstacles are placed and determines the overall Difficulty of Navigation through the Map.

The Static Environments are Procedurally Generated from a Given Set of Fixed Templates, and There is No Change in the Environment During Evaluation Therefore, This Makes These Maps Ideal to be Tested for Generalization and Robustness. These Maps Are Also Useful in Conducting Qualitative Evaluation of The Policies Because Each Map Will Provide A Unique Spatial Structure Which Will Provide Insight into The Policy's Operation Conditions Under Varying Constraints.

In Summary, the static terrain environment will supplement the randomized terrain environments for The Development of Robustly Evaluating Definitions of Locomotion and Towing Capability for Conditions in which Robots will Be Used in Highly Constrained, Obstacle-Rich Environments.

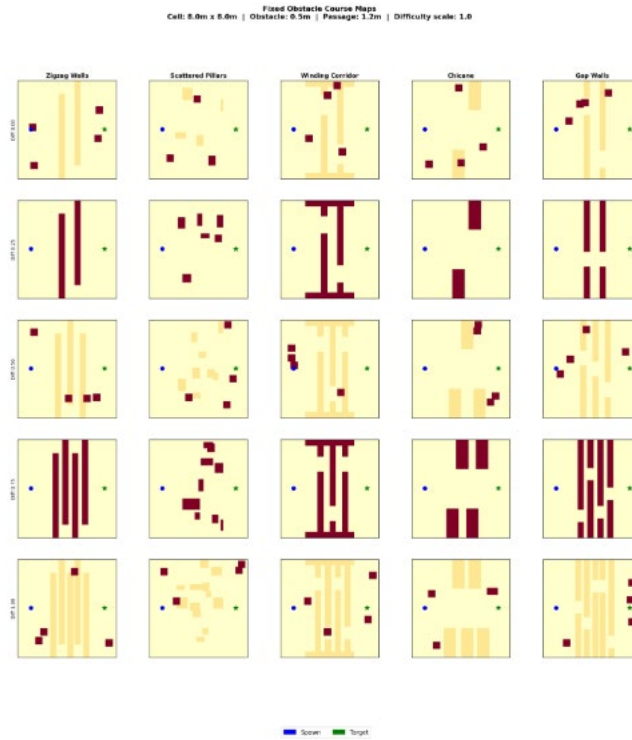


Figure 3.3: Fixed Static Terrain Configurations for Evaluation and Deployment

## Rewards & Penalties

### 1. Trot Contact Reward

This reward serves as an incentive for the establishment of a clean diagonal symmetric gait, as is typical for stable and efficient quadruped locomotion. In particular, it encourages the trot gait in which the diagonal pairs of feet (left front and right hind together) alternate in their contact with the ground. The rhythmic placing of diagonal foot pairs supports both stability and energy conservation while navigating over uneven terrain.

To monitor this activity on the robot, the actual contact pattern being made is continually being compared to an idealized set of two trot templates noted in the following: P1 and P2. These templates correspond to the two different alternating phases of a trot; one phase when the left front and right hind legs are touching the ground and the other phase when

the right front and left hind legs touch the ground. The robot's filtered contact state  $c_{filt} \in \mathbb{R}^4$  is used to measure deviation from these templates via the mean absolute error:

$$d_1 = \frac{1}{4} \sum_{i=1}^4 |c_{filt,i} - p_{1,i}|, d_2 = \frac{1}{4} \sum_{i=1}^4 |c_{filt,i} - p_{2,i}|$$

If both deviations are nonzero, i.e., the robot does not match either of the ideal patterns, the reward is suppressed. This is expressed using an indicator function:

$$match = 1[d_1 \cdot d_2 > 0]$$

To ensure the reward is only provided during meaningful movement, it is gated by the commanded velocity vector  $c_{x,y}$ . Specifically, the reward is only activated when the robot's commanded linear velocity in the plane exceeds a minimum threshold  $\delta$  (e.g., 0.3 m/s):

$$1[||c_{x,y}|| > \delta]$$

The final form of the reward function is:

$$r_{tot} = 1[d_1 \cdot d_2 > 0] \cdot 1[||c_{x,y}|| > \delta]$$

The formulation of rewards is necessary to establish policies for creating a natural locomotion pattern. By giving reinforcement for symmetry and alternating foot contact, the policy will develop smooth and stable gaits that can transfer to different types of terrain as well. The gating condition prevents the agent from receiving credit for incidental or unintentional contact configurations when stationary or moving slowly. This ensures that the reward aligns solely with purposeful and efficient movement.

The goal of designing this reward is to achieve an optimal balance between robustness and expressiveness. The use of filtered contact and simple pattern matching makes it lightweight to compute, yet effective in guiding complex gait learning. As the policy continues to be trained, it begins developing an internal representation of these diagonal templates, resulting in a locomotion that is similar to that of biological trotting (this is an essential prerequisite for deployment in the real world).

## 2. Velocity Reaching Reward

While directional alignment is critical for efficient locomotion, achieving appropriate translational speed is important, especially in tasks requiring responsive navigation or

payload transport. The intentional Velocity Reaching Reward is intended to encourage the quadrupedal agent to reach a specific speed whenever it is attempting a motion of any type. This reward will also allow an agent to learn how to employ acceleration and braking techniques more effectively.

### Mathematical Formulation:

Let  $V_{xy}$  be the robot's linear velocity vector on the horizontal plane. The reward is calculated as:

$$r_{velocity} = \begin{cases} \frac{\text{clip}(\|v_{xy}\|, 0, v_{max})}{v_{max}} & , \quad \text{if } \|c_{xy}\| \geq \delta \\ 0 & , \quad \text{otherwise} \end{cases}$$

where:

- $\|v_{xy}\|$  is the norm of the base velocity in the x-y plane,
- $v_{max}$  is the maximum expected speed (e.g., 3.0 m/s),
- $c_{xy}$  is the commanded speed vector, and
- $\delta$  is a small velocity threshold (e.g., 0.3 m/s) to suppress the reward during near-zero commands.

The reward value is normalized by  $v_{max}$ , ensuring that full credit is given only when the robot reaches or exceeds the target speed without surpassing it excessively.

### Design Justification:

The establishment of this reward not only has direct implications for directing the agent but also for controlling the rate of speed at which it moves and how effectively it can modulate its stride length and step frequency in relation to speed. Additionally, by masking the reward when there is little to no commanded velocity, we prevent the agent from being misincentivized to move during stationary goals or pause phases.

Through this training scheme, the agent develops a much finer-grained awareness of the context in which it is operating. It learns when to remain still and when to accelerate confidently. It also allows the creation of smoother motions over time, and it provides for an adaptable response of the controller to varying speed demands in multiple types of terrain and towing conditions.

### 3. Goal Reaching Reward (Continuous Proximity Incentive)

The achievement of target destinations (or goals) in autonomous navigation tasks is a key requirement for autonomous navigation systems. This type of reward provides an ongoing continuous and dense reward signal that motivates the robot to actively advance towards its specified goal location. In contrast to sparse binary rewards which are only activated when the robot reaches its exact destination, the guidance provided by the continuous reward function allows the reward signal to increase progressively as the robot approaches its destination, thus facilitating a better and more smooth convergence in training.

#### Mathematical Formulation:

Let  $c_{xy}$  denote the commanded velocity vector, which indirectly reflects the distance to the navigation target (e.g., goal position or box location).

The reward is calculated as:

$$r_{goal} = clip\left(1 - ||c_{xy}||, 0, 1\right)$$

Where:

- $||c_{xy}||$  is the norm of the command vector in the horizontal plane (x-y).

The value is clipped to the  $[0, 1]$  interval to ensure numerical stability and bounded gradient updates.

#### Design Justification:

This formulation treats the command vector  $c_{xy}$  as a proxy for remaining distance. A large command norm indicates that the robot is far from its goal, resulting in a low reward. Conversely, as the robot gets closer and the commanded velocity diminishes, the reward approaches to 1. This design offers several key benefits:

Dense Gradient Signal delivers continuous feedback throughout each episode. The continuous feedback provided throughout the duration of each episode significantly improves both learning speed and training stability, particularly in environments that are not only complex but also have many obstacles creating cluttered conditions. Also, learning



Proximity Awareness is achieved through the use of a smoothed distance-based approach. Using this approach allows robots to better understand how they will approach the target when they arrive and allows the robot to produce smooth trajectories and control its motion with higher accuracy during the final approach to the target.

Moreover, generalization is supported by this new form of reward structure, which does not rely on a binary success criterion. This means robots may generalize better to trajectories that were only partially successful. Therefore, policies can adapt to situations they have never experienced before.

To sum up, the purpose of this reward is to encourage robots to move toward their targets in a manner that allows them to make steady progress. By doing so, they can connect their low-level motion behaviors with the higher-level navigation goal of efficiently reaching their intended destinations.

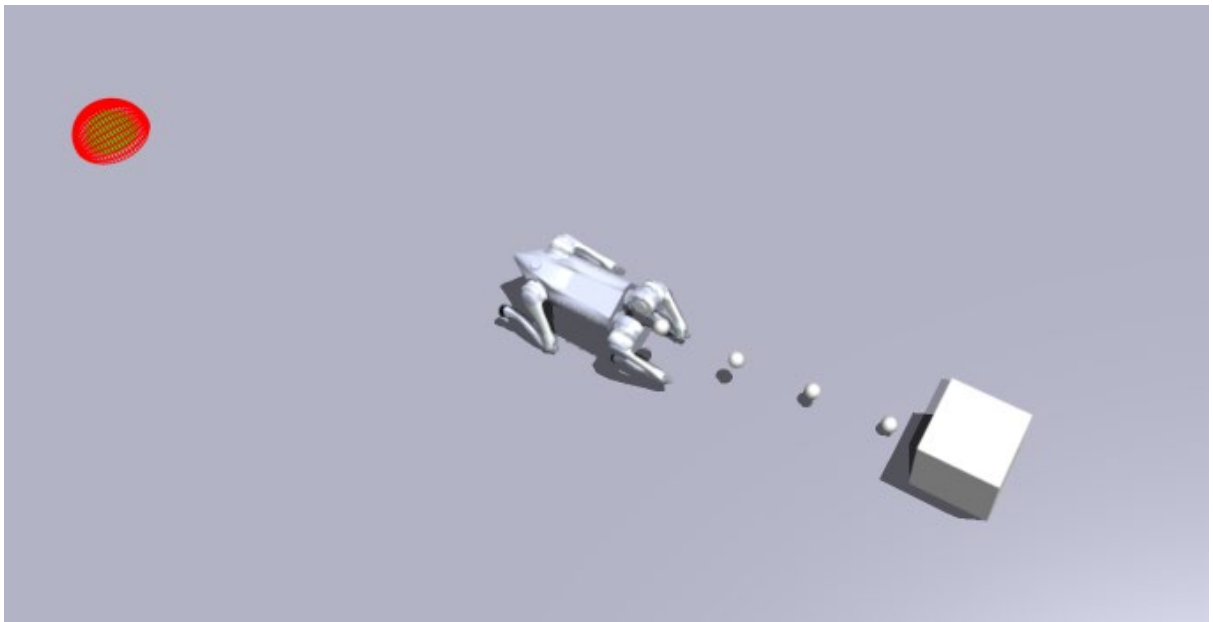


Figure 3.4: The quadruped robot moving steadily toward the goal marker while towing the box behind it, demonstrating successful goal-directed navigation

#### **4. Goal Completion Reward (Terminal Success Bonus)**

While constant proximate rewards help direct a robot to its desired target, a binary successful signal is used to reinforce complete task accomplishment. The Goal Completion Reward serves this purpose by granting a one-time reward only when the robot finishes an episode having reached the target within a specified threshold. This reinforces the agent's learning what a successful trajectory is and shapes its terminal behaviour so that it can be reliably deployed.

### Mathematical Formulation:

$$r_{complete} = \begin{cases} 1, & \text{if } d_{goal} < \delta \text{ and } t = t_{end} \\ 0, & \text{otherwise} \end{cases}$$

Where:

- $d_{goal} = ||cxy||$  be the norm of the commanded velocity vector, representing distance to the goal.
- $\delta$  a small distance threshold indicating goal proximity.
- $t_{end}$ : the last timestep of the episode.

### Design Justification:

This terminal reward offers a limited but strong incentive in addition to the continuous shaping signals. The design of the terminal reward satisfies several key objectives.

First it helps to Encourage Completion of the Goal. The agents receive the terminal reward only on episode completion, which instils in them that while they can get closer to the goal, they must precisely commit to it. Also, it can prevent agents from stopping too soon means they can stop once they think they have achieved their goal. Temporal gating ensures that agents do not stop close to the goal and exploit their current status. Rewards for proximity will not be available for agents until they remain in close proximity to the goal for the entire episode.

Futhermore, it Replicates both Local and Global Targets as continuous rewards encourage better local decision-making (e.g., velocity, timing), while a binary signal ensures that local decisions are aligned with the global task objective. Policies must learn both how to progress toward their overall task, and when they should cease progressing toward it. It also balances Local and Global Objectives. While continuous rewards improve local decision-making (e.g., direction, pacing), this binary signal ensures alignment with the global task goal. The policy must learn both how to progress and when to stop precisely.

In conclusion, the Goal Completion Reward reinforces consistency of episodes through training agents to accomplish their goals as opposed to reach them in proximity. This will create a more consistent and stable navigation behaviour when agents navigate through high-risk environments.

## 5. Goal Direction Reward (Trajectory Alignment Incentive)

The robot must move purposefully in a specified direction along its path, as well as being encouraged to get close to a goal and rewarded once the task is accomplished. The Goal Direction Reward gives the robot guidance to make its velocity vector point toward the commanded motion direction. When the robot points in a specific direction, it has a goal direction reward to help keep the robot from making inefficient random movements or drifting to the side.

### Mathematical Formulation:

Let:

- $c_{xy}$  denote the commanded goal vector in the x-y plane.
- $v_{xy}$  denote the robot's base velocity vector in the same plane.
- $\hat{c}$  and  $\hat{v}$  represent their normalized (unit) forms.

The cosine similarity is computed as:

$$\cos(\theta) = \hat{c} \cdot \hat{v} \frac{c_{xy} \cdot v_{xy}}{\|c_{xy}\| \cdot \|v_{xy}\| + \varepsilon}$$

To ensure robustness, we mask out low-speed noise by zeroing the reward when either the commanded velocity or the actual velocity is below a threshold. The reward is defined as:

$$r_{dir} = \begin{cases} 1, & \text{if } \cos(\theta) > 0.5 \\ 0, & \text{otherwise} \end{cases}$$

### Design Justification:

The Goal Direction Reward is essential for ensuring that the robot performs a movement that has the maximum efficiency and best alignment with locomotion behavior. It is not

only the speed of a robot's movements but also encourages a robot to maintain a straight trajectory toward its goal, instead of drifting laterally or zig-zagging. Also, it helps to increase the efficiency of navigation through cluttered or restricted environments by providing a large directional cone within which the robot can navigate. The Goal Direction Reward is very important when towing some type of load through narrow passages.

The reward system, based on the cosine similarity threshold (0.5), is designed to suppress spurious rewards. By only rewarding movements that are within a 60-degree cone of the final goal, this threshold allows for the filtering out of accidental alignments of movement caused by random or noisy movements. When used in conjunction with velocity magnitude, goal completion, and other reward signals, the directional reward aids in resolving many of the uncertainties present in the early learning stages of the agent (e.g., whether to move sideways or change orientation).

All in all, it is used to create a linear connection between low-level actions and higher-level plans, enables an agent's movements to not only complete the task of arriving at its designated location effectively, but also to do so in an efficient and goal-driven manner across the entire episode.

### **3.2.4 Observation**

In this project, we use the actor-critic algorithm. Actor makes an action based on observation and the critic evaluates the policy by estimating expected future rewards and safety penalties based on privileged observation.

Observation tensor in the project includes proprioception, terrain scan, box position, and privileged information. In addition, the information of the previous 10 steps is stacked in the observation tensor, providing historical information for making an action.

Proprioception contain vectors that is related the robot's base information, including base linear velocity, base angular velocity, projected gravity, commands, joint positions, joint velocities, and previous actions. The base linear velocity of the robot would not be observed by the actor, since the true value is difficult to measure in real deployment.

The terrain scan contains the binary value of 187 sampled points on the terrain distributed across a grid in the robot's surroundings, which provides the environment information to the agent.

Box position provides the 2D local coordinates of the towed box relative to the robot base frame, which is a vector pointing to the position of the box.

Privileged observations are only passed to the critic and would not be observed by the actor. It includes foot contact information, simulated delay, mass parameter, terrain friction, restitution coefficients, motor strength, and PD gains. The information would be used to evaluate the action of the actor.

## Obstacle detection grid

The system uses a binary obstacle representation derived from terrain height measurements. At each step, the 187 height samples are compared against a threshold: if the terrain at a given point rises significantly above the expected ground level, that point is marked as 1 (obstacle); otherwise it remains 0. This gives the policy a simple, easy-to-interpret map of where obstacles are.

Importantly, this binary signal does not physically block the robot from moving. Instead, it serves two purposes. First, it is fed directly into the observation vector, so the policy can "see" which directions are obstructed. Second, it drives several reward and cost terms during training — the more obstacle points detected nearby, the larger the penalty. There is also a dedicated cost that discourages high-speed movement when obstacles are detected ahead. Through these soft penalties, the policy gradually learns to slow down, steer around, or carefully navigate through cluttered areas, rather than being hard-coded to stop or reverse when an obstacle appears.

In practice, this mechanism is organized into three layers. At the observation level, `heights_binary` (187 dimensions for the robot) is concatenated into the main observation buffer, while `box_heights_binary` (187 dimensions for the towed payload) is included in the privileged observations available only during training.

In regard to cost, the penalties are more directed: `_cost_robot_obstacle_dist` and `_cost_box_obstacle_distance` specifically address the forward points of contact (i.e., obstacle points in the robot's environment) which are weighted according to their distance from the robot. `_cost_limit_vel_when_obstacle` penalises any instance where the robot exceeds a velocity of 1 m/s while having obstacles classified as detected in front of it. As a group, these three layers enable the policy to have both the capacity to perceive and the incentive to navigate through obstacles smoothly.

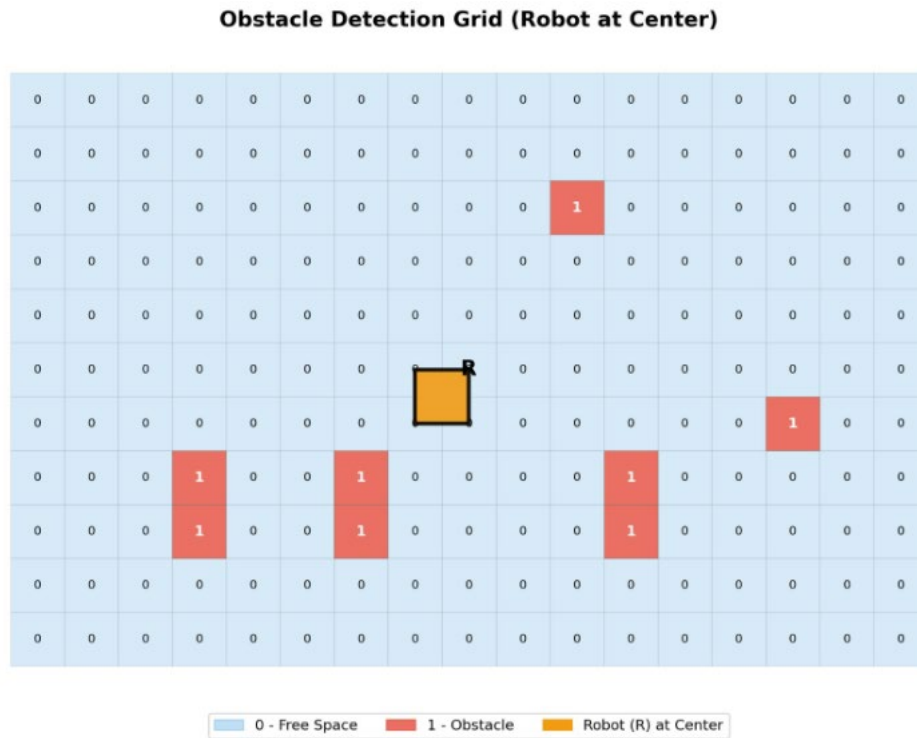


Figure 3.5: A binary obstacle detection grid where cells marked with 1 represent obstacles and cells marked with 0 represent free space around the robot

The primary benefit of the height-based binary obstacle encoding is that it is consistent and temporally stable in nature. As direct measurements of terrain are taken at each time step, the system provides a defined and repeatable signal at each time step. The structured form of representation provides the policy with a spatially located (geographically relevant) understanding of the environment that is vital for maintaining balance and preventing collision. It is important during payload towing, where safe navigation requires anticipating nearby obstructions.

This system enables the robot to perceive the environment and adjust speed and plan safe footholds. The combination of this obstacle-aware locomotion framework and the structured penalties and cost terms enables the agent to learn intelligent behavior that is sensitive to terrain during both the training phase as well as in their deployment environment.

### 3.2.5 Payload Towing System

The payload towing system constitutes a primary element of this project as it provides the quadruped robot with the ability to interact and transfer a payload between the two robots within complicated environments. In contrast to approaches that consider the box as an independent kinematic object updated by procedural rules, the box is directly embedded into the URDF model of the robot as part of the articulated chain. This section explains how to complete a URDF-based model of the box and how to create the connection between the box and the robot, how to manage the physics of towing and the reward functions created to enforce proper towing behavior.

#### URDF Based Towing Model

The towing mechanism is defined in the URDF file `go2_with_box.urdf`, which extends the standard robot model by appending a chain of links and joints. A series of revolute joints, such as `joint_base_to_rope`, `joint_point1`, `joint_point1_point2`, `joint_point2`, `joint_point2_point3`, `joint_point3`, ..., `joint_point5`, and finally `joint_point5_to_box`, connect the robot's base to the box. Each intermediate link (`rope_point_1_1`, `rope_point_1_2`, etc.) has a small mass and inertia, and each joint is assigned damping and friction coefficients to mimic the behavior of a flexible rope.

This URDF model provides the ability for Isaac Gym to simulate complete dynamics of the towing system, including tension transmission, lateral sway, and collision response, while removing the need for custom kinematic methods of updating these properties. The joint limits of this URDF are set at  $\pm\pi$ , giving approximately unconstrained rotational freedom. By incorporating the box into the robot as part of its asset, the environment benefits from consistent collision detection and contact force calculations for all rigid bodies.

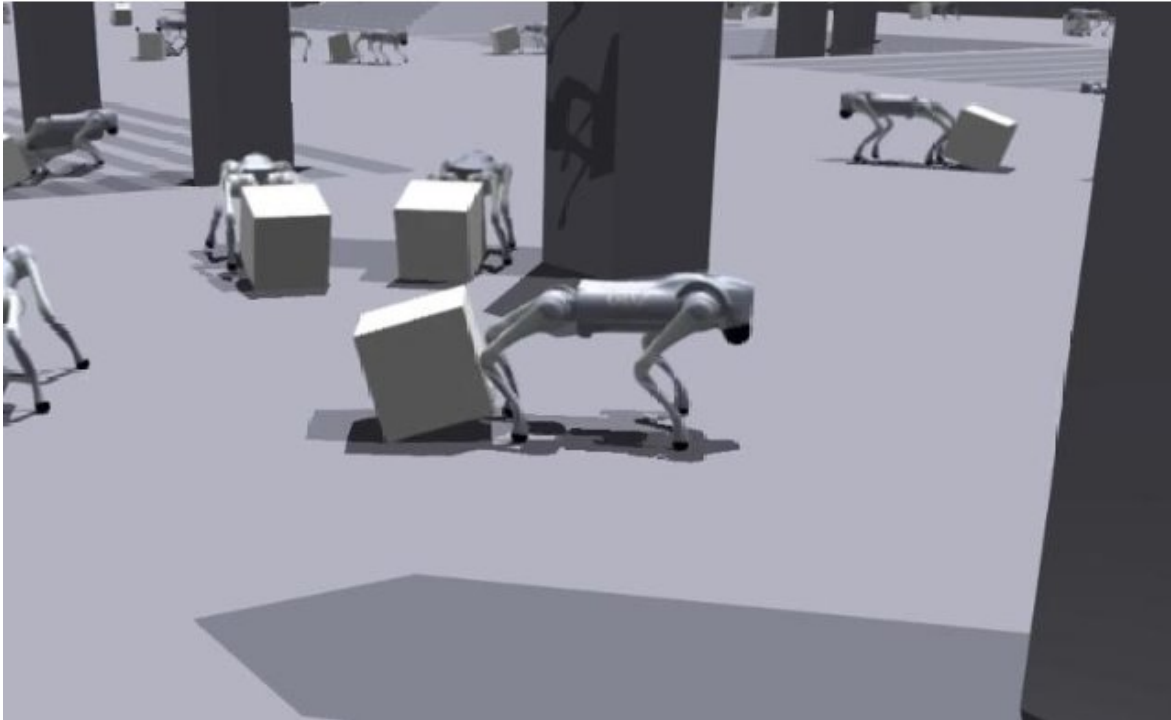


Figure 3.6: The quadruped robot successfully tows a box-shaped payload behind it while navigating the environment

## Payload Integration and Asset Loading

During environment creation, the `_create_envs` function produces an instance of the URDF asset using the function `self.gym.load_asset`, which reads in the asset path constructed from the configuration. The asset includes both the robot and the tow chain. Thus, the box and rope segments will be instantiated with the robot when creating an instance of each environment. The code will then search for the box in the environment using `self.cfg.asset.box_name` and record the index of the box's rigid body within the `rigid_body_states` tensor to allow for later retrieval of the box's position and orientation.

The DOF (Degrees-of-Freedom) for each of the rope joints is accounted for in the DOF state tensor. However, the joint "motors" for the rope and leg joints are modeled differently. In `_init_buffers`, the PD gains associated with the rope joint motors are set explicitly to zero, indicating that they will not experience motor torque but will instead be driven entirely by external forces and their own dynamics. This method of modeling the rope joints as passive



devices is appropriate since the cable is a tow cable that can only transmit tension and compression through the articulated chain when subjected to an external load.

## Physics Handling and Observation Extraction

The physics simulation advances the entire articulated system, including the rope and box, through the standard Isaac Gym pipeline. After each simulation step, the `post_physics_step` method refreshes all state tensors. The box's position and orientation are extracted from `self.rigid_body_states` using the stored `self.box_index`. Additionally, the vector from the robot to the box in the robot's body frame is computed as `self.box_vec_body` and included in the observation buffer, providing the policy with direct awareness of the box's relative location.

No explicit kinematic pulling or orientation correction is applied to the box. Its motion emerges solely from the physics of the articulated chain and the robot's actions. This approach preserve realistic interaction, including the box's ability to swing, rotate, and collide with obstacles, while avoiding the complexity of designing a custom kinematic model.

## Reward and Cost Signals for Safe Towing

Two feedback signals were implemented to guide the policy toward safe towing: a reward for avoiding robot-box collisions, and a cost for box-obstacle proximity. Both are weighted equally with a factor of 0.1 in the configuration.

### 1. Robot-box collision avoidance (reward)

The function `_reward_no_robot_box_collision` compute a binary reward: it return 1.0 if the minimum distance between the box's root position and any rigid body of the robot exceed 0.15 m, and 0.0 otherwise. This reward is multiplied by the scale 0.1 defined in `self.reward_scales["no_robot_box_collision"]`. By providing a positive signal only when the box remains separated from the robot, the policy learn to avoid sharp turns and maneuvers that could cause the box to strike the robot itself.

$$Reward \propto \mathbf{1}_{\{\min_{k \in B_{robot}} \|position_{box} - position_{robot}\| > d_{safe}\}}$$

Where:

- $B_{robot}$  denotes the set of rigid bodies belonging to the robot,
- $d_{safe} = 0.15m$  is the minimum distance between the robot and box,
- $\mathbf{1}_{\{\cdot\}}$  is the indicator function.

## 2. Box-obstacle collision avoidance (cost)

The function `_cost_box_obstacle_distance` enforce a penalty when the box approach terrain obstacles. It first compute a binary mask `self.box_heights_binary` indicating where terrain heights exceed an obstacle threshold (0.3 m). This mask is then multiplied element-wise by `self.reward_mask` (which assign higher importance to points closer to the robot) and by a forward-direction mask that only retain obstacle points in front of the robot. The resulting masked sum is returned as the cost. This cost is multiplied by the scale 0.1 (`self.cost_scales["box_obstacle_distance"] = 0.1`). By focusing on obstacles ahead, the cost selectively penalize situations where the box is about to collide with forward obstacles, encouraging the policy to steer clear of hazardous regions while ignoring obstacles behind.

$$Penalty \propto \sum_{i \in P} \left( \mathbf{1}_{\{height_i > height_{obs}\}} \cdot w_i \cdot \mathbf{1}_{forward}(i) \right)$$

Where:

- $P$  is the set of terrain sampling points around the box,
- $height_i$  is the terrain height at point  $i$ ,
- $height_{obs} = 0.3m$  is the obstacle height threshold,
- $w_i$  is the element of `self.reward_mask` (higher weight for points near the robot),
- $\mathbf{1}_{forward}(i)$  helps to only select the points lying in the forward direction of the robot

Together, these signals serve as the primary safety supervision for the towing task. The reward signal helps with basic self-separation, while the cost signal prevents the box from moving into an area with many obstacles. It may also be possible to add a reward for remaining in the center-trailing position and a cost for excessive lateral motion later in future improvements.

### 3.3 Evaluation Metrics and Validation Plan

After training in simulated environments, the policy is evaluated on the same terrain in training. The evaluation would consider the reward signal during training, the success rate of reaching the target, and the success rate of avoiding obstacles.

In reinforcement learning, the reward function is designed to guide the agent's learning process. It tells what actions are desirable. Analyzing changes in the reward signal over time, we could know whether the agent learns an effective policy and achieves the goal.

In the training, the “goal\_reached” and “goal\_reaching” reward functions have unstable average signals and failed to converge. The “goal\_reached” reward is a sparse reward, it is given when the distance between the box and the target is less than 0.3 meters at the end of an episode. In contrast, “goal\_reaching” provides a dense, continuous reward, which is also based on the distance between the box and target, it gives higher rewards for smaller distances. Throughout training, the average values of these two components fluctuated around 0.05 and 0.008, respectively. This possibly suggests that the robot’s policy remains unstable in consistently reaching the target.

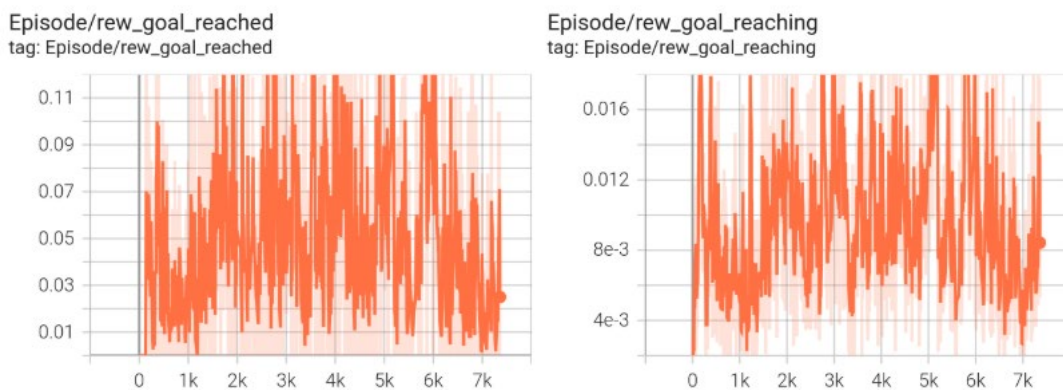


Figure 3.7: Reward diagrams of rew\_goal\_reached and rew\_goal\_reaching

Furthermore, cost terms are used to penalize the robot for getting too close to obstacles. The cost term “cost\_robot\_obstacle\_dist” has a higher penalty when the robot is close to obstacles, while “cost\_box\_obstacle\_distance” has a higher penalty when the box is close to obstacles. During training, the average “cost\_box\_obstacle\_dist” decreased slightly from 0.385 to 0.36. Meanwhile, “cost\_robot\_obstacle\_distance” decreased from 0.06 to 0.035

by episode 1000, but later rose again to approximately 0.05 by episode 7000. These may indicate that the policy may have a trade-off between robot collision and box collision.

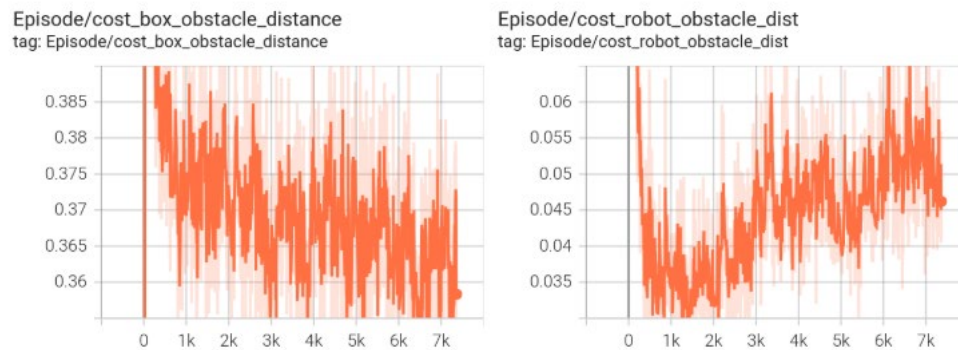


Figure 3.8: Cost diagrams of cost\_box\_obstacle\_distance and cost\_robot\_obstacle\_dist

In addition to reward signal analysis, the trained policy is evaluated through simulation on the same terrains used during training.

In the simulation, terrain is separated into 50 sub-terrains, each sub-terrain contains 3 to 6 obstacles, with difficulty increasing across rows. A total of 50 robots, each towing a box as a payload, are tested for 10 episodes. In every episode, the target point is randomly placed between 6 and 8 meters from the robot. The success threshold is set as 1, where success is defined as the box reaching within 1 meter of the target. Collision is recorded separately for the robot and box if they are within 0.15 meters of an obstacle.

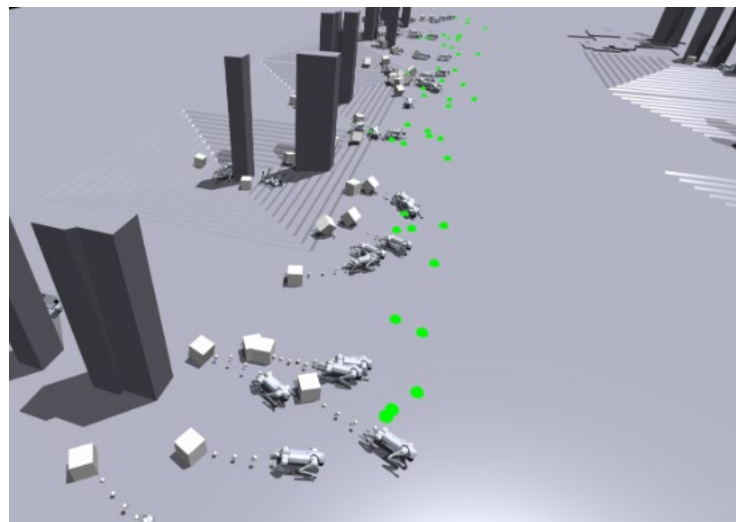


Figure 3.9: Simulation of evaluation process on training terrain

The policy achieves an average target-reaching rate of 75%, and the obstacle avoidance rate is 95% and 98 % for the robot and box, respectively. Visual inspection of the simulation shows that robots generally follow navigation commands, successfully detect nearby obstacles, and redirect their path accordingly.

However, it is noticed that the policy still has some limitations. For instance, when the robot is walking the stairs, it could lose balance when trying to avoid obstacles. It highlights that balancing goal-directed navigation, obstacle avoidance, and stable locomotion may be difficult. Furthermore, since the robot only observe terrain height within a maximum range of 0.15 meters, the robot could get stuck between closely spaced obstacles, particularly when the target is placed near them.

Apart from the original terrains, we have developed a more complex and challenging terrain for evaluation. The fixed-form environment has five layouts that are replicated in the horizontal direction. Obstacles (walls, narrow passages, chicanes, and pillars) are strategically placed in each layout to limit the robot's movement, thereby allowing evaluation of its navigation skills. The vertical axis of the environment suits progressively more challenging environments, where each level of progression increases with the adjustable variable representing the number of obstacles and their complexity within the given layout.

Unlike the randomized and curriculum terrains used during training, these static terrains do not change from one episode to another. hence, enabling reproducible benchmarking and more rigorous evaluation. We decide to set the obstacle height above the robot's stepping capability (e.g., 0.5 m), to ensure that collisions must be avoided rather than overcome. Each of the layouts has also been designed with a minimum width of the passage that is slightly greater than the robot in order to facilitate movement through the layout, while at the same time creating challenges in terms of foot placement, control of orientation, and coordination while towing.

The fixed terrain is useful for determining the generalization and robustness qualifications for how well a device can perform on different types of terrain. The robot cannot utilize a learned solution to negotiate the terrain so must be capable of reasoning through the geometry of its obstacles, adjusting its locomotion and pathing accordingly (especially if towing a payload that also has to avoid collision with the obstacle).

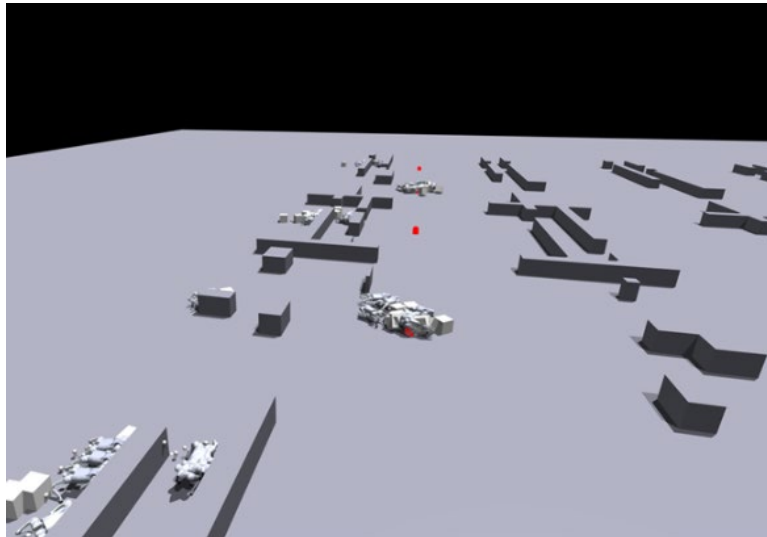


Figure 3.10: Simulation of evaluation process on challenging terrain

After running the simulations, the success rate dropped significantly to 45%. Most robots are too close to obstacles during turns, causing the box to get stuck. This performance can be attributed to the limited sensing distance of the grid map, while robots were not trained on such complex terrain.

### 3.4 Validation Plan

In validating the trained obstacle avoidance and goal reaching policy for the quadruped robot, the validation phase will include actual physical deployment of the trained policy onto the quadruped robot to evaluate its effectiveness in performing obstacle avoidance and goal reaching in a real operational environment. This verification will determine how well the policy that was trained via simulation will successfully transfer to a physical robot and provide a measure of how well the trained policy performs to achieve goals in physical environments as related to similar training conditions. The verification will use the Unitree Go2 quadruped robot with an onboard camera and Jetson Orin NX computing module, which is the same hardware that was assumed during development of the trained policy.

### Experimental Setup

The robot will be deployed in an indoor environment with obstacles that are arranged to resemble the sparse and cluttered arrangements found during the robot simulated training environment. Obstacles can consist of lightweight material such as cardboard boxes, foam blocks, or other materials that pose no risk to the robot or surrounding individuals from a collision. The surface of the ground will be flat so that navigation performance of the robot and towing performance of the robot can be isolated from any issues that may arise from the robot adapting to the terrain, which will be tested in a separate research project in the future. When tested, the robot will have a specific start position and an approximate distance of 3-5 meters from its starting position will be a pre-determined goal location which will have obstacles placed along the line directly between those locations, necessitating the robot to perform purposeful avoidance manoeuvres.

## Perception and Localization

The environmental perception aspect of the capabilities of the robot relies upon the depth camera to capture information about obstacles within the environment and to process this information into binary grid format in the same manner as the robot was trained to do. This allows the robot to maintain consistency between simulation and real-world perception pipelines. The robot will determine the position of the box by using an AprilTag attached to the surface of the payload to help identify its location. The camera will read the AprilTag and derive a distance to the box from the scale of the AprilTag as seen by the robot. This approach permits accurate relative position information to be derived without the need for external motion capture systems. To track the robot's trajectory for analysis, Ultra-Wideband (UWB) sensors will be placed around the test area to track the robot's position and orientation.

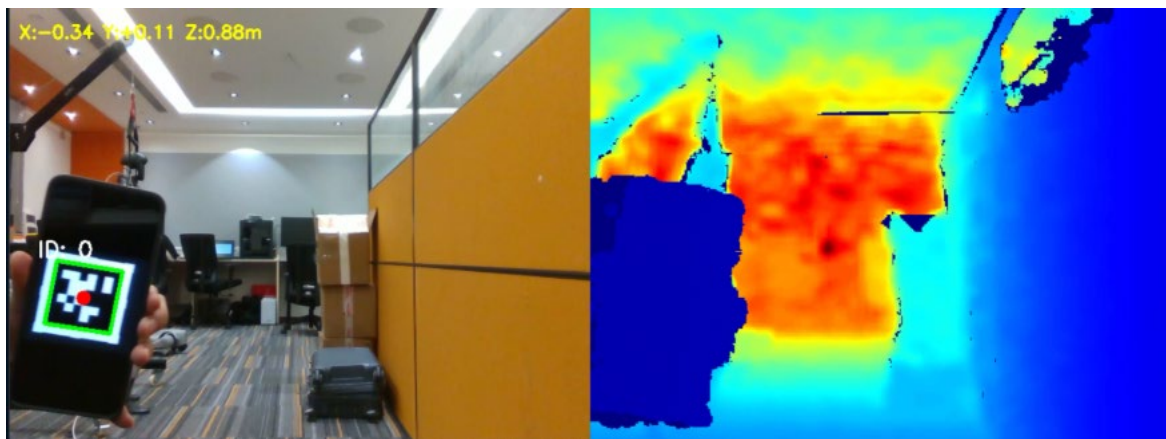


Figure 3.11: An example of the position (x,y) values and April tag ID are calculated and recorded

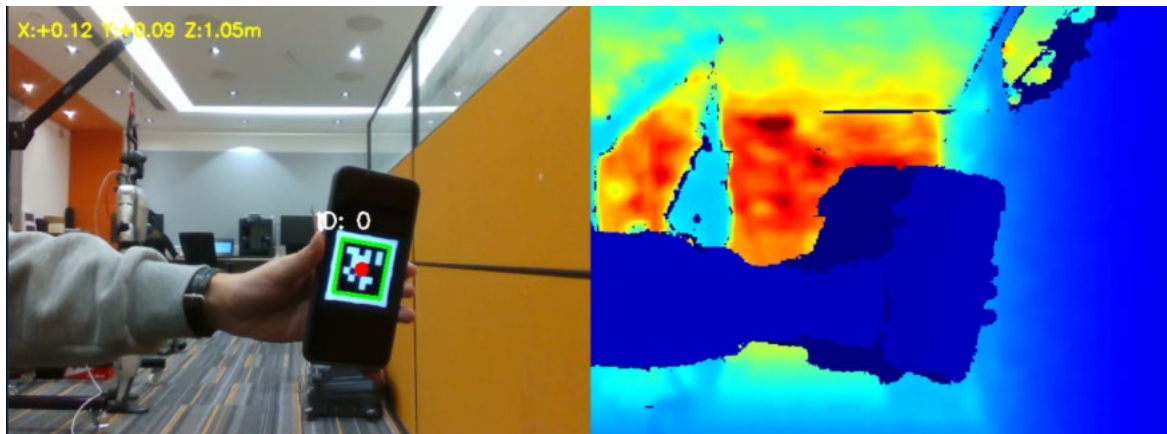


Figure 3.12: An example of the position values are changed as the April tag is moved

## Experimental Procedure

The validation will consist of a slalom path utilizing an obstacle configuration that contains alternating obstacles requiring turning. Each trial will involve the robot being commanded to navigate to a goal while towing a box. A successful trial is defined as a robot reaching the goal in the allotted time (60 seconds) without tipping over or having collisions and damages for the box. The position of the box in relation to the robot will be logged using AprilTag tracking in each trial, allowing for quantitative analysis of lateral offset and following behavior.

## Data Collection and Metrics

The following metrics will be recorded for each trial, such as navigation success (binary), time to goal, number of robot-obstacle collisions, number of box-obstacle collisions, minimum distance between robot and box, and maximum lateral deviation of the box from the robot's heading. These metrics will be aggregated from 30 trials to calculate success rate, average collision frequency, and dispersion statistics. Furthermore, the inference time



and memory usage will also be recorded by the onboard computer to confirm that the binary grid map process operate within the resource constraints of the Jetson Orin NX.

## **Analysis and Comparison**

The actual performance in deployment with simulation results under the same obstacle configuration will be compared. By analyzing differences in success rate, collision rate, and box localization, the root causes of the performance gap between simulation and reality, such as unmodeled friction, sensor noise, or actuator delay could be identified. Video clips from successful and failed trials will also be recorded for further analysis and provide a insights for improving the domain randomization parameters. Validation results help to determine whether the proposed framework possesses the robustness required for practical deployment.

## 4. Experiments and Results

### 4.1 Ablation / Sensitivity Analysis

To evaluate the robustness of policy, we performed a sensitivity analysis by varying different key parameters and analyzing their impact on overall performance. The analyzed variables include the command resampling interval, target-reaching threshold, collision detection threshold, and box mass. This analysis helps identify which parameters the policy is most sensitive to and provides insights of stability.

#### 4.1.1 Sensitivity to Command Resampling Interval

The command resampling interval refers to the time duration between the command updates. It is tested at values of 0.1, 1, 5. Results show that a lower resampling interval is positively correlated with the target-reaching success rate, which means a shorter resampling interval has better performance. This is because the command is a directional vector from the robot frame pointing toward the target position. When the resampling interval is too large, the robot operates with outdated direction information, which loses track of the correct direction and fails to reach the target in time.

| Resampling interval(second) | Target-reaching rate |
|-----------------------------|----------------------|
| 0.1                         | 75%                  |
| 1                           | 68%                  |
| 5                           | 7%                   |

Table 4.1: Sensitivity analysis for different command resampling intervals

### 4.1.2 Sensitivity to Target-reaching Threshold

The target-reaching threshold refers to the maximum allowable distance between the payload and the target position for a trial to be considered successful. It is tested at values of 0.3, 0.5, and 1.0. Results show that thresholds of 0.3 m and 0.5 m lead to significantly lower target-reaching success rates. This indicates that the robot is unable to tow the payload sufficiently close to the target position under these tighter thresholds.

| Target-reaching threshold(meters) | Target-reaching rate |
|-----------------------------------|----------------------|
| 0.3                               | 20%                  |
| 0.5                               | 38%                  |
| 1.0                               | 75%                  |

Table 4.2: Sensitivity analysis for different target-reaching thresholds

### 4.1.3 Sensitivity to Collision Detection Threshold

The collision detection threshold refers to the minimum distance between the payload and obstacles, which is considered a collision. It was tested at values of 0.15m, 0.3m, and 0.5m. The results show that obstacle avoidance performance drops significantly as the collision detection threshold increases. This indicates that the robot operates with the payload positioned very close to obstacles during execution.

| Collision Detection Threshold<br>(meters) | Obstacle avoidance rate<br>(robot ) | Obstacle avoidance rate<br>(box ) |
|-------------------------------------------|-------------------------------------|-----------------------------------|
| 0.15                                      | 96%                                 | 98%                               |
| 0.3                                       | 50%                                 | 59%                               |
| 0.5                                       | 30%                                 | 44%                               |

Table 4.3: Sensitivity analysis for different collision detection thresholds

### 4.1.4 Sensitivity to Box Mass

The box mass refers to the mass of the payload being towed. Values of 0.5 kg, 1.0 kg, and 2.0 kg are tested. The results show similar performance across all tested box masses, indicating that box mass is not a sensitive variable for the policy.

| Box Mass (Kg) | Target-reaching rate | Obstacle avoidance rate (robot) | Obstacle avoidance rate (box) |
|---------------|----------------------|---------------------------------|-------------------------------|
| 0.5           | 75%                  | 96%                             | 98%                           |
| 1.0           | 72%                  | 94%                             | 97%                           |
| 2.0           | 74%                  | 95%                             | 98%                           |

Table 4.4: Sensitivity analysis for different box masses

# 5. Discussion

## 5.1 Limitations and Practical Implications

The current framework consists several limitations that will be considered and proceed for future development. Although binary grid representations are computationally efficient, they cannot encode height information. Robots cannot distinguish between obstacles of different heights, which may result in poor navigation performance on obstacles with low clearance or on sloping terrain that requires adaptive gait adjustments. The evaluation on complex and challenging terrain further confirm this limitation. When tested on more difficult terrain not encountered during training, the success rate dropped from 75% to 45%, with most failures due to the robot positioning itself too close to obstacles during turns, causing the box to become stuck.

The training process reveal that certain reward components failed to converge. The sparse reward “goal\_reached” and the dense reward “goal\_reaching” showed unstable average signals throughout the training, the value is fluctuating around 0.05 and 0.008 respectively, suggesting that the policy remain unstable in consistently reaching the target. Additionally, the cost terms reveal a potential trade-off between robot and box collision avoidance. While “cost\_box\_obstacle\_distance” decreased slightly from 0.385 to 0.36, “cost\_robot\_obstacle\_distance” decreased from 0.06 to 0.035 by episode 1000 but later rose to approximately 0.05 by episode 7000, indicating that the policy may be balancing between protecting the robot and payload.

The payload configuration is restricted to a single box geometry with fixed range of mass and size. If irregular or deformable payloads are implemented to this policy, it may require retraining and additional reward terms to account for different interaction dynamics. Furthermore, the traction connection is achieved through virtual kinematic constraints, rather than through physical cables or joints. While this approach simplify the simulation, it gives an unrealistic behavior, which is that the robot and the string can pass through obstacles as the string was intangible, but the box remain subject to collision detection. Therefore, the robot may bypass the obstacle while the box is stuck behind the obstacle. Since the virtual connection does not physically interact with the environment to pull the box through the same gap. This may lead to overly optimistic performance in the simulation, which does not fully reflect the challenges of physical deployment.

Finally, the training and evaluation was conducted on terrains with obstacles and moderate complexity, achieving an overall target-reaching rate of 75% and obstacle avoidance rates

of 95% for the robot and 98% for the box under standard conditions. However, the simulation videos show that the policy still exist issues in some situations. When traversing stairs, the robot occasionally lose balance while it attempt to avoid obstacles, highlighting the difficulty of balancing goal-directed navigation, obstacle avoidance, and stable locomotion. Also, the robot can get stuck between obstacles that are very close together, especially when the target is near them. Steep slopes, rugged landscapes, and complex terrain resulted in a success rate of 45%. With these factors were not adequately addressed in the training courses, the application of the policy in such extreme terrain need further training and adjustments.

## 5.2 Practical Implications

The proposed framework has multiple real-world deployment advantages. Most notably due to their relatively low computational footprint, the binary grid representation is deployable on numerous systems, many of which are embedded, without high-performance processor. This is a key feature for field robotics as the onboard computational resources of many field robots are limited by the power and weight of the robot itself. This binary perception pipeline that relies entirely on thresholded depth data, is particularly amenable to low-cost depth sensors available through commercial quadraped platforms, thereby further lowering the hardware barrier to adoption. Through experimental evaluation of our simulation, our framework demonstrates a target reaching rate of 75% across 50 robots, each tested through 10 episodes with 95% of all obstacle avoidance for the robot and 98% for the box, indicating that the perception and control pipeline is practical and robust for moderately complex deployment scenarios in real-world environments.

Using a single agent for logistic transportation is ideal for many scenarios in which multiple robots coordinating together would not be possible, such as in disaster relief situations where many times the areas that need assistance are very confined because of either narrow hallways, collapsed buildings, areas that have lost communication or areas that are not large enough to support many robots. A single agent will effectively move objects through obstacles on terrains that difficult to traverse. Simulation studies show that the robot are unfamiliar with crashing into obstacles and is able to navigate effectively through obstacles, maintaining an overall collision rate of 5% with other robots and 2% with boxes and other objects. Likewise, another area that could be used with this system is for delivering construction materials in the construction industry on work sites that have uneven terrain or delivering products in commercial facilities through cluttered aisles.

This framework is designed to be extensible to other types of payloads with minimal modification. The current implementation supports a box shaped payload with fixed mass. However, the reinforcement learning training pipeline can be used with other types of payloads that use different payload geometries and mass properties by re-training with new simulation parameters. The reward and cost functions used to define the general safety of an operation use general principles of safety rather than heuristics associated with specific types of payloads so they would remain valid for multiple types of payloads. However, as the performance on complex, untrained terrain, had a drop in performance to less than 45% the results show that training would be needed to extend the approach to work in different environments. This gives rise to the ability of the approach to be adapted to handle irregular shapes of transport like toolboxes, containers and sensor packages with appropriate retraining on the specific terrain type to be used. As a result future deployments could use this framework to support various logistics operations in many different types of operational domains so long as the environment for deployment falls within the distribution of the training conditions.

## 6. Individual Technical Contributions: Lee

### Chung Hei

#### 6.1 Overview of Responsibility

My contributions included both theory and practice through performing literature reviews and implementing . I reviewed current and seminal literature to identify key concepts and had the subsequent literature review developed the conceptual and technical basis for the project. Using this information, I created a grid-based obstacle identification and detection system that allowed for spatial awareness of both the robot and the towed payload by converting terrain height samples into structured binary maps. Additionally, I assisted in the co-development and refinement of numerous locomotion reward functions that were designed to assist with the towing of payloads, ensuring a stable gait and safe foot placement while traversing challenging terrain. To provide baseline evaluation performance metrics and assessment of how well the system could be generalized, I developed and implemented a fixed-map terrain testing infrastructure that consisted of procedurally created terrain layouts that allowed for varying levels of difficulty and obstruction arrangement. Lastly, I played a key role in transitioning the technology to a real-world application by designing and testing a real-time depth box detection algorithm that used edge contour algorithms and geometric filtering to identify the towed payload in real-time video feeds, and to produce an estimate of the box's position in relation to the policy acting on the real-time input from the camera.



## 6.2 Implementation Details

### Implementation Overview

In order to ground our work in the most up-to-date research, I collected and synthesized a wide variety of peer-reviewed literature. From this, I identified several key trends, such as moving away from purely proprioceptive controllers to controller architectures that hierarchically integrate visual input, creating hybrid planning/learning systems (e.g., Deep Tracking Control), and identifying a large gap in research regarding single-agent payload towing in the presence of obstacles. These trends specifically impacted our architectural choices, such as creating dense observational spaces, developing task-specific reward functions, and employing structured terrain as a means for systematically assessing performance.

### Grid-Based Terrain Perception

For each robot and its payload, the terrain perception module creates 187 height value samples in a circular area around the robot. These vertical height measurements create a binary obstacle map. Any height value below a threshold value is considered clear or traversable ground. The binary obstacle representation does not limit robotic movements, but is used by the policy as part of the perception process and the training process. The binary map is embedded in the observation vector, allowing the policy to perceive obstacle locations. This representation is also used as part of the training process to learn the appropriate movement away from obstacles and develop policies to minimize or eliminate collisions. The terrain perception binary representation includes two layers of feedback to provide information to the robot for navigation: First, general penalties based on proximity to obstacles are applied; Second, additional cost terms apply when the robot is travelling forward through obstacles or travelling at a high speed in cluttered areas. The combination of these two types of signals will lead to the robot learning to travel slower, steer away from, or navigate carefully around obstacles and not to rely on a hard-coded stopping behavior to avoid collisions.

## Reward Functions

I developed a reward function design for locomotion with towing that promotes balance and efficiency in complex environments. The system uses diagonal foot placements and clean foot placements as positive reinforcements for creating a symmetrical trot while offering penalties on lateral impacts and off-center swing movements to reduce stumbling. Other penalty terms are included for stalling, which are areas where robots are commanded to go somewhere, but do not make any progress forward. Stalling occurs frequently when a robot is towing a heavy load in tight or narrow spaces. Other positive reinforcement terms promote correct foot time in the air, clearance of obstacles and rhythm/repeating foot movements on various terrains. The challenge with tuning these terms is making small changes can create instability in the gait or lead to overfitting. However, through iterative testing, analysis and adjustments of the variables, I was able to create variables that would provide for consistent, repeatable towing on all terrains.

## Fixed Obstacle Maps

In order to examine navigation strategies, I built a set of maps with a size of  $8 \times 8$  meters each with fixed obstacles. These maps have zigzag routes; a variety of pillars placed in random locations; meandering routes that curve back on each other; sharp bends at right angles; and narrow openings with obstacles approximately 0.5 m tall that will prevent the robot from going straight through them. As such, the robot will need to choose other, alternate pathways through the passages. The passages are slightly wider than the robot's width, allowing them to be utilizable by the robot, while still requiring precision for navigating through. Although these maps were initially created as benchmark tests to evaluate generalization, because of their deterministic nature, these maps are also an excellent resource for developing a laser-focused training regimen around learning how to follow corridor layouts and validating how the robot's navigation strategies will perform in the deployment stage. These maps give me a means of creating controlled environments in which to systematically determine the correlated failure modes; to evaluate the robotics' resilience; and to make corrections to policies to implement in real-world applications.

## Deployment preparation

The development of a box detection module utilizing the Intel RealSense D435 depth camera in order for the system to be deployed in the real world. The system segments the depth map into discrete layers based on height differences, then applies morphological operations to refine edges and identifies candidate regions that match shallow, rectangular box surfaces. Each candidate region is assessed using several different types of filtering to evaluate how well the area matches the expected size and shape of a rectangular box. Additionally, an optional color-based detection (e.g. HSV space) can also be applied to help increase robustness. The box detection module produces an output that is a measurement of the lateral position and distance of the closest valid detection and includes a confidence score. While the accuracy of the box detection module is less than that of fiducial markers such as AprilTag and similar products, the detection method still provides an excellent closed loop testing mechanism with real-time feedback. This enables a researcher or engineer to take their simulated trained policy into the physical world without relying upon external tracking devices.

## 6.3 Critical Analysis

### Reward Function Optimization

Optimizing and refining the reward system for towing was the most time-consuming and critical aspect of the project. Robots also experience dynamic coupling between the payload and the robot during the process of towing; therefore, small changes in weighting of the reward terms will frequently result in unforeseen consequences. Many reward terms are interdependent, including foot air time, clearance, and direction of movement. As a result, changing one of the reward terms will require rebalancing of multiple other terms as well.

An example of this would be increasing the penalty for tripping or having a poor foot position, it was common to see overcompensation from both legs with one leg raising higher than normal or pausing in mid-swing. This was not usually observed until multiple rounds of training had taken place so the project could only be debugged through an iterative process. The process of pinpointing which reward was responsible for causing some instability or inefficiency would involve repeatedly reviewing both video rollouts and reward breakdown logs as well as trajectory visualisations.

To tune penalties for stalling correctly, there were two things: if they were too low, there would be no stalling penalty effect at all; if they were too high, they would create false positives that would penalise a robot that moved slowly when close to the goal. All of the reward signals were actively receiving reward signals at the same time, and therefore the management of the combination of all of the various reward signals was the greatest challenge as opposed to simply designing the individual terms. Due to the need to evaluate side effects, it was important to conduct full testing and training of the entire system after every change was made.

Through this type of approach, we have identified both the interdependent relationship between reward shaping and complex tasks and how we can systematically and empirically tune these rewards, using both visual and statistical feedback to validate progress.

# 7. Conclusion and Future Work

## 7.1 Summary

This research project developed an integrated reinforcement learning framework for using a single quadraped robot to transport a weighted vehicle through a challenging, cluttered environment. This project aims to overcome some of the challenges of existing multi-agent approaches to payload transportation by developing an integrated approach that focuses on the benefits of autonomous, single-agent towing through cluttered environments; a solution that requires a high level of locomotion robustness, control stability when carrying dynamic loads, and navigation awareness of the environment.

To address this challenge, we started by developing a high-throughput simulation platform, which was Legged Gym that was customized to create both new working environments for the robot as well as terrain generation systems. The robot model used in our simulations was based upon Unitree Go2 and trained with a Constrained Proximal Policy Optimisation (CPPO) method. The CPPO method allowed for a unique combination of maximising rewards whilst including explicit safety costs into the optimisation process. By combining these two areas of focus, we were able to create a simulation environment in which an agent could optimise performance and learn behaviours to safely avoid violations of kinematics, as well as falling or colliding with the towed box.

The introduction of a grid-based terrain perception module was a technological breakthrough on this project, enabling the conversion of the 3-dimensional terrain data into a structured, low-dimensional format suitable for reinforcement learning (RL) policy training. This allowed for more efficient perception and awareness of obstacles while minimizing the computational burden of the system. In addition, a kinematic model was developed to simulate the payload to avoid creating unstable physical dynamics during training and to maintain the training stability needed for developing RL policies while still capturing the essential difficulties associated with towing.

A major influence on the robot's learned behaviour results from how reward functions are designed. The robot received rewards for aligning its movements toward target objectives, avoiding collisions with obstacles (both itself and its cargo) and maintaining a smooth and efficient gait. Rewards for specific foot behaviours were introduced to encourage the proper timing of swing phase, encourage sufficient vertical clearance during swing phase and smooth/consistent foot placement resulting in leg movements that are expressive in their own way and adapt to changes in terrain.

Through extensive simulations of various difficulty levels of procedurally generated terrains, the validity of this framework was confirmed. Ablation and sensitivity analyses have also revealed the expected contribution of each component of each reward as well as the various design decisions that are made in designing our final policy. Our final policy showed a high level of performance over many different terrains with a consistent degree of accuracy when reaching their goal and experiencing a low frequency of falling and few total box collisions—achieving all criteria established originally.

To conclude, the findings of this study showed that a single quadrupedal robot could be trained with real-time payload towing capabilities and in unstructured environments using deep reinforcement learning based on constraints. It represents a step forward in expanding the operational capabilities of legged robots beyond basic locomotion toward more practical and mission-oriented applications.

## **7.2 Future Work**

While the project establishes a strong foundation for single-agent payload transport via reinforcement learning, several areas remain open for improvement and further investigation. These extensions aim to strengthen real-world applicability, generalization, and system robustness.

### **1. Sim-to-Real Transfer**

All evaluations and training associated with this project were made using the Isaac Gym backend and all training and evaluations took place in simulation. While simulations allow for high levels of environmental control and parallel rollouts, one major hurdle in transferring policies produced in a simulation to a physical quadruped robot are the many differences between simulated and real-world dynamics. Unmodeled effects such as latency on actuators, variations in traction from multiple surface types, noise introduced by contact with the environment, and hardware limitations will not be found in the simulated environments. Realistic sensor noise, delay, and lack of complete accuracy mean that the information exposed to these sensors is much less accurate than what was available during training where perfect access to the modelled state was provided.

To address this, domain randomization techniques could be introduced to improve robustness. This includes randomizing mass properties, terrain friction, actuation gains, or perception noise during training. By incorporating these variations into the learning process, the developed policy will generalise across a wider range of distributions of environments. Focusing on physics fidelity can also be accomplished through the use of high-fidelity

simulators, such as Mujoco or NVIDIA's Isaac Sim, which implement control systems that reflect the constraints placed upon hardware. Techniques such as residual learning or fine-tuning using real-world data may also be utilised to adapt the developed policy to the real world without recurrently training from the ground up.

## **2. Payload Modeling and Complexity**

In this project, we assume that a rigid object (a box) is attached to the robot with basic movement patterns. This simplifies the process of learning how to navigate under load; however, true-world objects can be movable like carts, not rigid like bags, hoses, or possibly change shape or behaviour over time. The different types of load will have a major impact on balance, forces acting on the load, and how the robot moves. As such, the ability to change the load density, shape, and movement during the training process would allow the robot's training process to be more applicable to real-world situations.

While navigating through a cluttered area with a load, the robot must also take into consideration the movement path of the load in addition to its own position in space. By providing additional information about what is going on between the load and surrounding environment, for example, terrain sampling of boxes around the robot or predicting future box positions, we can prevent boxes from being dragged into walls or obstacles. In many applications, such as disaster response or confined transportation, this type of reasoning is critical for success.

## **3. Enhanced Terrain Perception**

The grid-based Terrain Sensor module improves the resolution and consistency of the spatial perception when compared to previous methods of sampling the environment. However, this method also assumes that an agent has full knowledge of the terrain around it based on synthetic height maps. In real deployment, terrain information must be acquired from sensors such as depth cameras, LiDAR, or stereo vision, all of which introduce partial observability and noise.

Future work will include incorporating actual sensor pipelines into the observation module of the agent using either simulation of real sensor input or a learned encoder that converts sensor perception data into structured terrain representations. Using a combination of proprioception and exteroception (as has been done by recent hierarchical Reinforcement Learning studies) can enable reactive and adaptive gait modulation in a wider variety of dynamic and unstructured terrains through long-range terrain predictions. Long-range terrain predictions may give agents more time to prepare for obstacles and plan accordingly.

## 4. High-Level Planning and Multi-Scale Control

This project focuses on low-level policy learning, which is generating joint torques and body-level actions based on immediate perception and local goals. This type of policy learning is very effective for very short-term policies and controls but to perform tasks such as navigating a robot from one location to another, for example, from location A to location B in a large building or a collapsed building, requires higher-level planning as well as policy learning.

The next step in our work will be to combine the locomotion controller that has been learned with a global planning system. This will involve hierarchical architectures that uses a high-level policy to provide subgoals or behaviours, for example, "climb stairs", "turn left", "get to a secure exit", for the low-level controller to safely perform. The use of a modular architecture will also allow mission specific optimizations based on the terrain type, payload limitations, and timeliness of the mission.

## 5. Safety and Constraint Adaptation

The primary purpose of CPPO was to create a method to apply physical safety constraints to policies in the learning process. In this project, the physical safety costs, such as slipping, instability, and stalling, were represented by soft penalties that were factored into the learning process. The constraint weights and thresholds remain fixed and there was no explicit modelling of recovery strategies.

In future work, it may be possible to develop adaptive or learned constraint mechanisms. For example, the ability to change penalty values based upon the amount of trust that the system has, the risk level of the terrain, and the condition of the actuator. Also, the recovery policies can be developed, such as regaining balance from a disturbance, and emergency stops. Using uncertainty-aware policy optimization methods or safe exploration algorithms, like shielding and predictive risk estimating, during training and deployment may help avoid failure.



# Bibliography

1. Achiam, J., Held, D., Tamar, A., & Abbeel, P. (2017). Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 22–31). PMLR.
2. Altman, E. (1999). *Constrained Markov decision processes*. Chapman & Hall/CRC.
3. Chen, W.-T., Nguyen, M., Li, Z., Sue, G. N., & Sreenath, K. (2025). Decentralized navigation of a cable-towed load using quadrupedal robot team via MARL. *arXiv preprint*.
4. Gangapurwala, S., Mitchell, A., & Havoutis, I. (2020). Guided constrained policy optimization for dynamic quadrupedal robot locomotion. *IEEE Robotics and Automation Letters*, 5(2), 3642–3649.
5. Hoeller, D., Rudin, N., Sako, D., & Hutter, M. (2024). Extreme parkour with legged robots. In *2024 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 11443–11450). IEEE.
6. Jenelten, F., He, J., Farshidian, F., & Hutter, M. (2024). DTC: Deep tracking control. *Science Robotics*, 9(86), eadh5401.
7. Ji, Y., Zhang, B., & Sreenath, K. (2021). Reinforcement learning for collaborative quadrupedal manipulation of a payload over challenging terrain. In *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)* (pp. 899–904). IEEE.
8. Kim, J., Fawcett, R. T., Kamidi, V. R., Ames, A. D., & Hamed, K. A. (2023). Layered control for cooperative locomotion of two quadrupedal robots: Centralized and distributed approaches. *IEEE Transactions on Robotics*, 39(6), 4728–4748.
9. Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2020). Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47), eabc5986.
10. Makoviychuk, V., Wawrzyniak, L., Guo, Y., Lu, M., Storey, K., Macklin, M., Hoeller, D., Rudin, N., Allshire, A., Handa, A., & State, G. (2021). Isaac Gym: High performance GPU-based physics simulation for robot learning. *arXiv preprint*.
11. Miki, T., Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2022). Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62), eabk2822.

12. Rudin, N., Hoeller, D., Reist, P., & Hutter, M. (2022). Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning* (Vol. 164, pp. 91–100). PMLR.
13. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint*.
14. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 23–30). IEEE.
15. Yang, C., Yuan, K., Zhu, Q., Yu, W., & Li, Z. (2020). Multi-expert learning of adaptive legged locomotion. *Science Robotics*, 5(49), eabb2174.

# Appendix

## A.1 GitHub repository of the project

[https://github.com/EdmondNM/final\\_year\\_project](https://github.com/EdmondNM/final_year_project)